

Academic Year :- 2025-2026

MORYA STATIONERY WEBAPP

A Project Report

Submitted in partial fulfillment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Riddhi Rajesh Gurav

Seat No:- 3010497

Under the esteemed guidance of

Prof. Gufran Kabil Qureshi

Assistant Professor

&

Prof. Raviprakash Jaiswal

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

A.B.V.M. AGRAWAL JATIYA KOSH'S

G.D. JALAN COLLEGE OF SCIENCE, COMMERCE & ARTS

(Affiliated to University of Mumbai)

MUMBAI, 400097

MAHARASHTRA

Ghanshyamdas Jalan College of Science & Commerce

ISO 9001 : 2015 Certified

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN No.: 2024016400004684

Roll No. 73

1. Name of the Student

Ms. Riddhi Rajesh Gurav

1. Title of the Project

Morya Stationery WebApp

2. Name of the Guide

Prof. Gufran Qureshi

3. Teaching experience of the Guide _13 Years

4. Is this a live project? ☒ Yes

No ☐

If Yes, Name of the company:

Morya Stationery & Xerox

Signature of the Student

Date:

Signature of the Guide

Date:

Signature of the coordinator

Date:

A.B.V.M. Agrawal Jatiya Kosh's
G.D. JALAN COLLEGE OF SCIENCE, COMMERCE & ARTS
(Affiliated to University of Mumbai)
MUMBAI-MAHARASHTRA-400097

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project entitled, "**MORYA STATIONERY WEBAPP**", is bonafied work of **Ms. Riddhi Rajesh Gurav** bearing Seat No:3010497 submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide

Coordinator

External Examiner

Date:

College Seal

MORYA STATIONERY & XEROX

Shop No. 3, 4 & 62 , Shivdarshan Shopping Centre, Old Mhb Colony,
Gorai Road, Borivali West, Mumbai - 400091.

CALL : +91 87888 38525 /+91 97735 18477



TO WHOMSOEVER IT MAY CONCERN

This is to certify that Riddhi Rajesh Gurav , a student of G.D. Jalan College of Science, Commerce, and Arts, pursuing T.Y.B.Sc. L.T. for the academic year 2025-2026, has successfully completed the project "Morya Stationery WebApp" as per the requirements prescribed by Morya Stationery & Xerox Shop.

She has demonstrated dedication technical proficiency throughout the project, ensuring the system's functionality and efficiency. Her contributions to development, testing, and implementation have met our expectations.



ABSTRACT

The Stationery Shop Website is a full-stack web application designed to facilitate the online purchase and management of stationery products. Built using Node.js for the backend, MongoDB for the database, and a responsive HTML/CSS/JavaScript frontend, the application allows users to register, login, browse products, place orders, and make payments securely. The system efficiently stores and manages user information, product details, and order histories in a MongoDB database. This project demonstrates the integration of dynamic frontend interfaces with a robust backend, providing a seamless and user-friendly online shopping experience. It is ideal for small businesses seeking to digitize their stationery sales and streamline order processing.

ACKNOWLEDGEMENT

“The acknowledgment of a single possibility can change everything”

I have a great pleasure while keeping this Project work in front of reputed personalities, in the research. I take this opportunity as a deep sense of gratitude to those people & institutions that helped me during this research work.

Firstly, I would like to express my heartfelt gratitude **Dr. Rajeshwari Govindappa**, the Principal and **Dr. Rajeshwari Ravi**, the Director of G. D. Jalan College of Science, Commerce and Arts, who guided me throughout the research work and writing up of the thesis. Without her constant inspiration, encouragement and constructive criticism, the present work would not have been possible.

I take this opportunity to express my profound gratitude and deep regards to my guide **Mr. Raviprakash Jaiswal**, for exemplary guidance at each step of this project, monitoring and constant encouragement throughout the course of this project. He also helped me in preparing for a professional environment, where I realized the power of schedule.

I would also like to thank the department of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)**, for their resources and training that made the implementation smoother. A special mention to all the faculties, lab assistants and the non teaching staff of the department as well for their efforts and cooperation.

Lastly, I would like to thank the institute, **G. D. JALAN COLLEGE OF SCIENCE, COMMERCE AND ARTS**, for giving me this opportunity to explore and expand our horizons through this project and all our academic endeavor.

DECLARATION

I hereby declare that the project entitled, “**MORYA STATIONERY WEBAPP**”done at **MUMBAI**, where the project is done, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY) to be submitted as final semester project as part of our curriculum.

Riddhi Gurav
Name and Signature of Student

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....	1
1.1 Background.....	1
1.2 Objectives.....	2
1.3 Purpose, Scope, Applicability.....	2
1.4 Achievement.....	4
1.5 Organisation of Report.....	4
CHAPTER 2 : SURVEY OF TECHNOLOGIES.....	5
2.1 Existing System.....	5
2.2 Proposed System.....	5
2.3 Feasibility Study.....	6
2.4 Comparison With Technologies.....	9
CHAPTER 3: REQUIREMENTS AND ANALYSIS.....	16
3.1 Problem definition:.....	16
3.2 Requirements specification.....	16
3.3 Planning and Scheduling.....	19
3.4 Software and Hardware Requirements:.....	23
3.5 Preliminary Product Design.....	24
3.6 Conceptual Models.....	27
CHAPTER 4 : SYSTEM DESIGN.....	31
4.1 Basic Modules.....	31
4.2 Data Design.....	33
4.3 Procedural Diagram.....	37
4.4 User Interface Design.....	43
4.5 Security Issues.....	46
4.6 Test Cases.....	48
CHAPTER 5: IMPLEMENTATION AND TESTING.....	50
5.1 Implementation Approaches:.....	50
5.2 Code Details and Efficiency.....	53
5.3 Testing Approach.....	71
5.4 Test Cases.....	75
5.4 Test Cases Table.....	75
CHAPTER 6 : RESULTS AND DISCUSSION.....	76
6.1 Test Reports.....	76
6.2 User Manual.....	78
Chapter 7 : CONCLUSION.....	88
7.1 Conclusion.....	88
7.2 Limitations of the System.....	89
7.3 Future Scope.....	90
REFERENCES.....	91

LIST OF TABLES

2.4.1 Comparison of Frontend Technologies	9
2.4.2 Comparison of Backend Technologies	10
3.3.2 PERT Table	22
4.2.1 User Schema	33
4.2.2 Products Schema	33
4.2.3 Cart Schema	34
4.2.4 Order Schema	34
5.Payment Collection	34
4.2.5 Payment Schema	34
4.6.1 Registration test case	48
4.6.2 Login test case	48
4.6.3 Products order test case	49
4.6.4 Logout test case	49
5.1.2 Increment Model Approach Table	52
5.3.6 Comparison between Alpha and Beta Testing	74
5.4 Test Cases Table	75
6.1.1 Test Reports Table	76
6.1.2 Test Execution Table	76

LIST OF FIGURES

Figure 3.3.1 GanttChart for the First HalfProject -----	20
Figure 3.3.1 GanttChart for the Second HalfProject -----	20
Figure 3.3.3 PERT Diagram -----	22
Figure 3.6.1: E-R Model of Project -----	27
Figure 3.6.2 Use Case Diagram -----	28
Figure 3.6.3 DataFlow Diagram -----	29
Figure 3.6.4 Sequence Diagram -----	30
Figure 4.3.1 Logical Diagram of Project -----	37
Figure 4.4.1 Home Page -----	43
Figure 4.4.2 Login Page -----	43
Figure 4.4.3 Register Page -----	44
Figure 4.4.4 Products Page -----	45
Figure 5.1.1 Agile Model -----	50
Figure 5.1.2 Incremental Model -----	51
Figure 5.1.3 Scrum model -----	52
Figure 5.2.1 Home Page -----	54
Figure 5.2.2 Login Page -----	56
Figure 5.2.3 Product Page -----	63
Figure 5.2.4 Admin Page -----	65
Figure 5.2.5 Order Track Page -----	67
Figure 6.2.1 Register Page -----	78
Figure 6.2.2 Login Page -----	79
Figure 6.2.3 Product Page -----	81
Figure 6.2.3 Cart Page -----	82
Figure 6.2.4 Checkout Page -----	83
Figure 6.2.5 Orders Page -----	84
Figure 6.2.6 Tracking Page -----	85
Figure 6.2.7 Admin Panel -----	86

CHAPTER 1: INTRODUCTION

1.1 Background

In recent years, e-commerce has transformed the way people shop, offering convenience, speed, and accessibility. Traditional stationery shops, while providing essential products for students, offices, and businesses, often face operational challenges such as manual inventory management, order tracking, and maintaining customer records. Manual processes are prone to errors, time-consuming, and can lead to unsatisfied customers due to delays or mistakes in order handling.

The rise of web-based applications provides an opportunity to digitize stationery sales. By developing an online platform, shop owners can automate inventory tracking, manage customer orders efficiently, and expand their reach to customers who prefer shopping online. This also allows customers to explore products, place orders, and make payments without physically visiting the store, saving time and enhancing the shopping experience.

1.2 Objectives

1. **User Registration and Authentication:** Allow customers to create accounts and log in securely. Protect user data with proper authentication mechanisms.
2. **Product Browsing and Management:** Display a variety of stationery products with images, descriptions, prices, and availability. Enable easy addition, modification, or removal of products by the admin.
3. **Order Placement and Management:** Provide a smooth process for adding products to the cart and placing orders. Track order history for both users and admin.
4. **Payment Integration:** Implement secure online payment options to complete orders.
5. **Responsive and Interactive Interface:** Ensure the website works seamlessly on desktops, tablets, and mobile devices. Provide a visually appealing and intuitive user experience.
6. **Database Management:** Store all relevant information such as user credentials, orders, and product details in MongoDB for easy retrieval and management.

1.3 Purpose, Scope, Applicability

1.3.1 Purpose:

The purpose of the Stationery Shop web application is to provide a convenient, efficient, and secure online platform for both customers and shop owners. It aims to:

1. **Simplify the shopping process** by allowing customers to browse, select, and order stationery products online.
2. **Streamline business operations** by managing inventory, orders, and customer data digitally.
3. **Ensure secure transactions** through online payment integration.
4. **Enhance user experience** with a responsive and interactive web interface.

The project ultimately bridges the gap between traditional retail and modern e-commerce, improving accessibility and operational efficiency.

1.3.2 Scope

- **User Management:** Signup, login, and secure authentication.
- **Product Management:** Displaying product details dynamically with images, descriptions, and prices.
- **Order Management:** Adding items to cart, placing orders, and tracking order history.
- **Payment Integration:** Online payment for orders.
- **Database Management:** Storing users, orders, and product information in MongoDB.

1.3.3 Applicability:

The Stationery Shop web application is useful for small to medium stationery stores, educational institutions, and offices, allowing customers to browse, order, and pay online. It streamlines inventory, order, and customer management while providing a secure, responsive, and user-friendly platform. The system can also be adapted for other retail sectors, demonstrating scalability and versatility.

1.4 Achievement

1. Developed a full-stack web application using Node.js, MongoDB, and HTML/CSS/JavaScript.
2. Implemented secure user authentication (signup, login, logout).
3. Enabled dynamic product display and order management with history tracking.
4. Integrated online payment for smooth transactions.
5. Designed a responsive and user-friendly interface for all devices
6. Created a scalable system ready for future enhancements.

1.5 Organisation of Report

- **Introduction** – Provides an overview of the project, background, and purpose.
- **Objective** – Describes the goals and intended outcomes of the project.
- **Scope** – Defines what the project covers and its limitations.
- **Applicability** – Explains where and how the system can be used.
- **System Design & Architecture** – Details the overall system architecture, database design, and workflow.
- **Implementation** – Discusses the technologies used, modules developed, and coding aspects.
- **Achievements** – Highlights the accomplishments and functionality achieved.
- **Conclusion & Future Scope** – Summarizes the project and possible future enhancements.

CHAPTER 2 : SURVEY OF TECHNOLOGIES

2.1 Existing System

The current stationery store system is mostly manual, requiring customers to visit the store physically. Inventory, orders, and billing are managed by hand, leading to errors, delays, and inefficiency. There is no online payment or digital tracking, making the process inconvenient for both customers and store owners.

2.1.1 Problem With Existing System

- Manual record-keeping of inventory, sales, and customers is time-consuming and error-prone.
- Customers must visit the store physically to browse and purchase products.
- Order tracking and billing are inefficient and prone to mistakes.
- No online payment options, making transactions inconvenient.
- Inventory management issues like stock-outs or overstocking.
- Limited ability to analyze sales data or customer preferences.

2.2 Proposed System

The proposed system is a web-based Stationery Shop application that allows customers to browse products, place orders, and make secure online payments. It includes user authentication, dynamic product management, order tracking, and a responsive interface. The system uses MongoDB to manage data efficiently, improving convenience, accuracy, and overall shopping experience compared to the manual system.

2.2.1. Advantages of Proposed System

- **Efficiency:**Automates product management, order processing, and payments, reducing manual effort
- **Accessibility:**Customers can **shop online anytime and from anywhere** using desktops, tablets, or mobiles.
- **Accuracy:** Reduces errors in inventory, billing, and order tracking compared to manual systems.
- **Secure Transactions:**Online payment integration ensures safe and reliable financial transactions.
- **User-Friendly Interface:**Intuitive design improves customer experience and satisfaction.
- **Data Management:**Stores user, product, and order information efficiently in MongoDB, enabling easy retrieval and reporting.
- **Scalability:**Can be extended to include promotions, additional products, or analytics in the future.

2.3 Feasibility Study

A Feasibility Study Can Help You Determine Whether or Not You Should Proceed with Your Project. It Is Essential to Evaluate the Cost and Benefit of The Proposed System Types Of the Feasibility Study Are Taken into Consideration

- Operational Feasibility
- Technical Feasibility
- Economic Feasibility

2.3.1 Operational Feasibility

- **Improved Workflows :** The web application will integrate seamlessly into the company's existing workflows, including order management, production tracking, inventory control, and client communication. By automating key tasks, it will free up staff for higher-value work.
- **Ease of Use:** The application will feature a user-friendly interface, allowing staff, managers, and clients to quickly adapt to the system with minimal disruption to ongoing operations.
- **Customer Interaction:** Clients will also need a brief orientation on how to use the online order management system. This can be achieved through simple guides, tutorials, or customer support.

2.3.2 Technical Feasibility

The project is technically feasible as it uses widely available technologies like Node.js, MongoDB, HTML, CSS, and JavaScript, which are reliable, scalable, and well-supported. The required tools, frameworks, and hosting environments are easily accessible, ensuring smooth development and deployment.

2.3.3 Economic Feasibility

- **Operational Costs:** Once the system is implemented, the ongoing costs include cloud hosting fees, maintenance, technical support, and security updates.
- **Training Costs:** Employees will need training to use the system effectively, which might require a few sessions with a technology consultant developer.
- **Development Costs:** Additional costs may be incurred for hiring technical consultants or contracting a third-party development firm.

2.4 Comparison With Technologies

2.4.1 Nodejs vs Deno

Node.js is mature, widely adopted, and has an enormous ecosystem, making it a great choice for existing and large-scale projects. Deno offers a modern, secure, and developer-friendly alternative, especially for projects that want to leverage TypeScript, prioritize security, or prefer

Feature	Node.js	Deno
Language Support	JavaScript (TypeScript via third-party)	JavaScript (TypeScript via third-party)
Security	No built-in security model (all access is allowed by default)	Secure by default (explicit permissions required for file, network, or environment access)
Package Manager	Uses npm and yarn	Secure by default (explicit permissions required for file, network, or environment access)
Standard Library	No official standard library, relies heavily on third-party libraries	Comes with a built-in standard library for common tasks
Module System	CommonJS (require) and ECMAScript (ES) module	ES modules (import/export), uses modern ECMAScript modules by default
Dependency Management	package.json for dependency tracking	No package.json, imports from URLs directly

2.4.1 Comparison of Frontend Technologies

2.4.2 MySQL vs. MongoDB

Aspect	MySQL	MongoDB
Performance	Excellent for read heavy workloads	Good performance in specific cases
Ease of Use	Simple setup and documentation	Simpler for certain applications
Community Ecosystem	Large, active user community	Active community and ecosystem
Replication High Availability	Robust replication options	Replication options available
Compatibility	Compatible with many languages	Native drivers for languages
Dependency Management	package.json for dependency tracking	No package.json, imports from URLs directly

Table 2.4.2 Comparison of Backend Technologies

1. Nodejs

Node.js is a server-side runtime environment that allows JavaScript to be executed outside of a browser. Built on Chrome's V8 engine, it offers high performance and uses an event-driven, non-blocking I/O model, making it efficient for handling concurrent connections. Node.js is widely used for building web servers, APIs, and real-time applications, and it comes with npm (Node Package Manager), which provides access to a vast library of modules. It supports frameworks like Express.js and is well-suited for scalable applications, particularly in microservices and real-time systems.

Features of Nodejs:

1. Asynchronous and Event-Driven:

Node.js operates asynchronously, meaning tasks are executed in the background while the main thread continues processing other tasks. The event-driven architecture allows it to handle multiple operations without blocking.

2. Non-blocking I/O:

Node.js uses non-blocking I/O operations, which makes it ideal for I/O-intensive tasks like reading from databases or serving web pages without slowing down performance.

3. Single-Threaded but Scalable:

It follows a single-threaded model with event looping, which enables it to handle many connections simultaneously. The lightweight nature of Node.js means it can support a large number of concurrent connections.

4. Cross-Platform:

Node.js is platform-independent and runs on Windows, macOS, and Linux, which makes it versatile for deployment on various operating systems.

5. High Performance with V8 Engine:

Built on Google's V8 JavaScript engine, Node.js compiles JavaScript into native machine code, leading to faster code execution.

6. npm (Node Package Manager):

Node.js comes with npm, which is the largest ecosystem of open-source libraries. Developers can easily install, share, and update packages, speeding up development.

7. Microservices and API Support:

Its lightweight nature and modular approach make it a popular choice for building microservices and RESTful APIs.

Advantages of Nodejs:

- Fast Performance.
- Scalability.
- Single Programming Language.
- Real-time Applications.
- Rich Ecosystem.
- Cross-platform Development
- Community Support.
- Efficient for I/O-bound Tasks.
- Easy to Learn.
- Support for Microservices.

Limitations:

- Single-Threaded Model.
- Callback Hell.
- Not Suitable for CPU-bound Tasks.
- Mature Tools/Frameworks for Relational Databases

2. MongoDB

MongoDB is a popular NoSQL database known for its flexibility, scalability, and ease of use. Unlike traditional relational databases (RDBMS), MongoDB stores data in a flexible, JSON-like format called BSON (Binary JSON), making it suitable for applications that handle large volumes of unstructured or semi-structured data.

Features of MongoDB:

- NoSQL Database.
- Document-oriented Storage.
- Schema-less Design.
- Horizontal Scalability.
- High Availability with Replication.
- Indexing.
- Aggregation Framework.
- ACID Transactions.

Limitations of MongoDB:

- Lack of ACID Transactions (in Older Versions).
- High Memory Usage.
- Complex Joins.
- Limited Data Size in BSON Documents.
- No Support for Transactions at Scale.
- Consistency Trade-offs

FRONTEND TECHNOLOGIES

HTML

The Hypertext Markup Language or HTML is the standard markup language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript. HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered HTML. Advantages: There are numerous advantages to using HTML when creating a web page. Several examples include the following:

- Widely used programming language with a sizable community of users.
- Runs on every web browser.
- Utilizes a clean and consistent markup.
- Completely open-source and free.
- Completely open-source and free as with anything that has an advantage, HTML has a few negatives that you should consider as well

CSS

CSS, or Cascading Style Sheets, is a design language that simplifies making web pages presentable. Selectors are used to picking elements and applying styles to them. This is especially important for browsers like Chrome, Firefox, and Edge, designed to deliver documents to screens, printers, and projectors visually. CSS is widely used because it saves time, has easy maintenance, offers selectors, text effects,

- CSS is an extremely powerful style sheet language that is used to manage the appearance and feel of HTML content. CSS Ensures Design and Update Efficiency CSS enables us to construct rules and apply them to a variety of website elements. This strategy has numerous advantages when a client requests a site-wide update.
- Because the text is fully independent of the design, we can make modifications to our Style Sheet and have them take effect across all applicable instances. CSS Use May Result in Quicker Page Downloads Because the browser only downloads the rules once and then caches and reuses them for each page load, the adoption of CSS can result in faster page loads and enhanced performance. This results in a smaller server load and fewer requirements, which ultimately saves our clients' money.

JAVASCRIPT

JavaScript is a scripting language used to develop web pages. Developed in Netscape, JS allows developers to create a dynamic and interactive web page to interact with visitors and execute complex actions.

It also enables users to load content into a document without reloading the entire page. Most websites use JavaScript for validation and to support external applications, including PDF documents widgets, flash applications. Some of the world's largest tech companies use JavaScript to better the user experience. JavaScript might be a client-side scripting language, inferring that the client's browser handles ASCII (American Standard Code for Information Interchange) text file processing rather than an online server. With the aid of JavaScript, this can load the webpage without contacting the primary server For instance, a JavaScript function might verify that all the required fields are filled out on an online form before it is submitted.

The JavaScript code can output an error message before any data is sent to the server Though it supports the classic servers and systems, its growth is not parallel to the technological growth of the current industry standard.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem definition:

Traditional stationery stores rely on manual processes for inventory, billing, and order management, which leads to errors, delays, and limited accessibility. Customers must visit the shop physically, and there is no option for online browsing, ordering, or payment. This creates inconvenience for both customers and store owners, highlighting the need for a web-based Stationery Shop system that automates operations and provides a modern shopping experience.

3.2 Requirements specification

3.2.1 Existing system:

The existing stationery shop system is manual, requiring customers to visit the store for purchases. Inventory, billing, and orders are handled by hand, leading to errors, delays, and lack of convenience, with no online access or payment options.

3.2.2 Proposed system:

The proposed system is a web-based stationery shop application designed to overcome the limitations of the manual system. It allows customers to browse products online, place orders, and make secure digital payments. The system features user authentication, dynamic product management, order tracking, and a responsive interface. Data is stored in MongoDB, ensuring efficient management of users, products, and orders. The shop owner (admin) can easily add, update, or remove products and monitor orders. Overall, the proposed system provides a faster, error-free, and user-friendly shopping experience, improving both customer convenience and business efficiency.

3.2.3 System Requirements:

3.2.3.1 Functional Requirement:

1. User Management:

- User registration (signup) and login/logout.
- Secure authentication and session handling.

2. Product Management

- Display products with name, price, description, and image.
- Add, update, or delete products (admin).

3. Shopping Cart & Orders

- Add/remove products to/from the cart.
- Place orders and view order history.
- Admin can view and manage all orders.

4. Payment System

- Secure online payment integration for order completion.

5. Database Management

- Store and retrieve user, product, and order information in MongoDB.

6. Responsive User Interface

- Accessible on desktop, tablet, and mobile devices.
- Easy navigation and interactive design.

3.2.3.2 Non- Functional Requirement:

• Performance

The system should respond to user requests within 2–3 seconds under normal load.

• Scalability

The application should support an increasing number of users, products, and orders without major changes.

- **Security**

User credentials and payment details must be encrypted and protected. Only authorized users (admin/customers) should access respective features.

- **Usability**

The interface should be easy to navigate, even for non-technical users. Provide a clean and responsive design for multiple devices.

- **Reliability**

The system should be available with minimum downtime and ensure data consistency.

- **Maintainability**

Code should be modular and well-documented for easy updates and bug fixes.

- **Compatibility**

The application must work across modern browsers (Chrome, Firefox, Edge) and different devices.

3.3 Planning and Scheduling

The development of the **Stationery Shop Web Application** is divided into multiple phases to ensure systematic progress:

1. Requirement Analysis :

Identify system requirements (functional and non-functional)

Define scope, objectives, and feasibility study.

2. System Design:

Design system architecture, database schema, and user interface wireframes.

3. Frontend Development:

Build responsive pages using HTML, CSS, and JavaScript.

Implement navigation and basic UI features.

4. Backend Development:

Implement APIs with Node.js and Express.js.

Connect backend with **MongoDB** for user, product, and order management.

5. Integration & Payment Module :

Integrate frontend and backend.

Implement secure online payment system.

6. Testing Phase :

Perform unit testing, integration testing, and user acceptance testing.

Fix bugs and optimize performance.

7. Deployment & Documentation :

Deploy the application on server/cloud.

Prepare final project report and documentation.

Gantt Chart

A Gantt chart is a type of bar chart that illustrates a project schedule. This chart lists the tasks to be performed on the vertical axis, and time intervals on the horizontal axis. The Gantt chart shows planned and actual progress for several tasks displayed against a horizontal time scale. It is effective and easy-to-read method of indicating the expected and actual status for each of set of tasks compared to planned progress for each activity of the set. Gantt Charts provide a clear picture of the current state of the project.

The Figure below shows the expected and actual durations of the first half of the project timeframe :

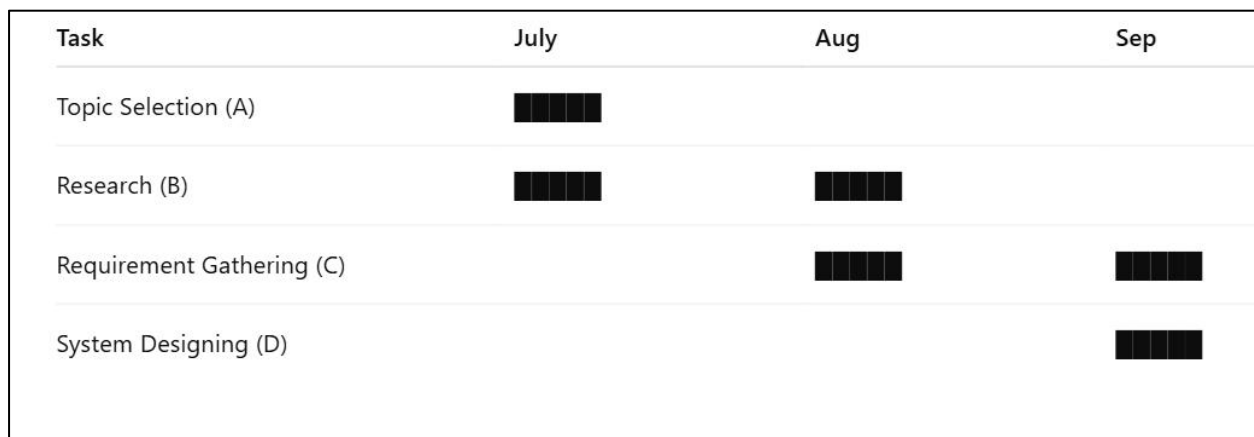


Figure 3.3.1 GanttChart for the First Half Project

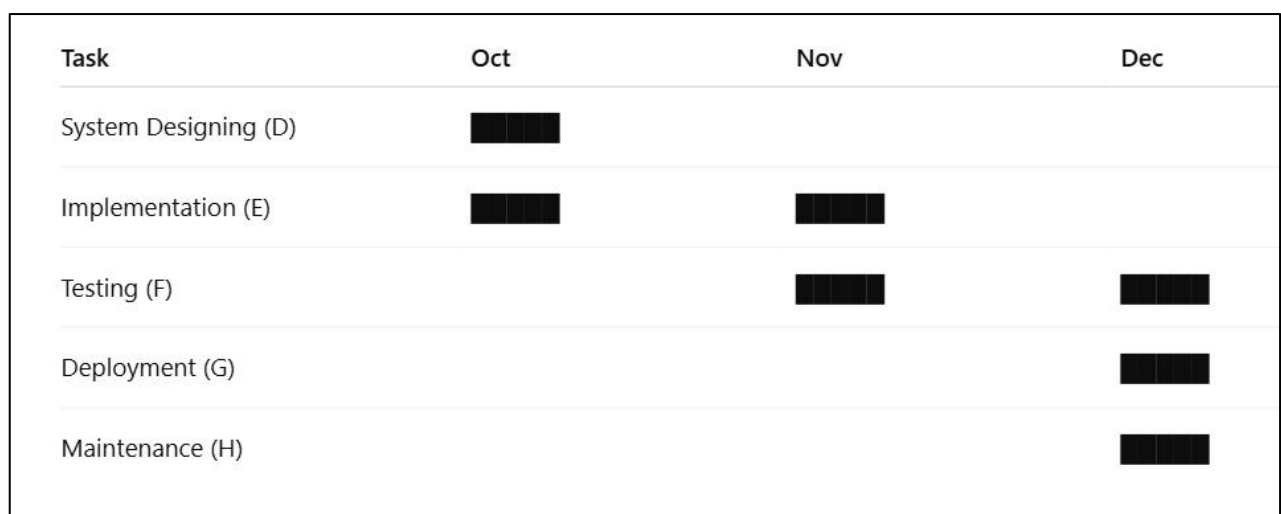


Figure 3.3.1 GanttChart for the Second Half Project

PERT Chart:

A PERT chart is a visual project management tool that's useful for mapping out project tasks and planning the overall project schedule. Many people confuse PERT with a PERT chart, so the best way to provide a comprehensive PERT chart definition is to start by clarifying both of these terms. PERT stands for Program Evaluation Review Technique. PERT is the actual technique used to create a PERT chart. Meanwhile, a PERT chart is the visual diagram that results from using PERT. Think of PERT as the process and the PERT chart as the outcome. PERT charts allow project managers to see essential scheduling details such as task dependencies, task duration estimates, and the minimum amount of time a project can be completed within. However, they're not the most user-friendly or wellunderstood tool. Difference between Pert chart and Gantt chart: Gantt Chart are a popular tool for visualizing project schedules. They're a type of bar chart that also shows task dependencies and project timelines. Gantt charts tend to be less technical and easier for viewers to understand as they don't include the same level of detail as PERT charts. For this reason, Pert charts tend to be better for initially estimating project timelines, while Gantt charts are often preferred for team project time tracking and keeping stakeholders up-to- date during the project.

Advantages and disadvantages of pert chart pert chart advantages include:

- Highlighting task dependencies and your critical path.
- More accurate estimates than when relying on single task duration estimates.
- Easy to see how long non-critical tasks can be delayed without impacting the project end date.

PERT chart disadvantages include

- Durations are still based on estimates. If you did a poor tour coming up with your numbers, this method won't be any more accurate than the critical path method.
- PERT charts can be time-consuming and difficult to create and update.
- They can be confusing for stakeholders to interpret.
- PERT assumes one task must end before the next starts. Other types of dependency relationships won't work well with this technique

PERT Table

Task	Duration (Days)	Activity	Depends On
Start Project	–	Start	None
Topic Selection	15	A	None
Research	20	B	A
Requirement Gathering	25	C	B
System Designing	30	D	C
Implementation	35	E	D
Testing	20	F	E
Deployment	10	G	F
Maintenance	25	H	G

3.3.2 PERT Table

PERT Diagram

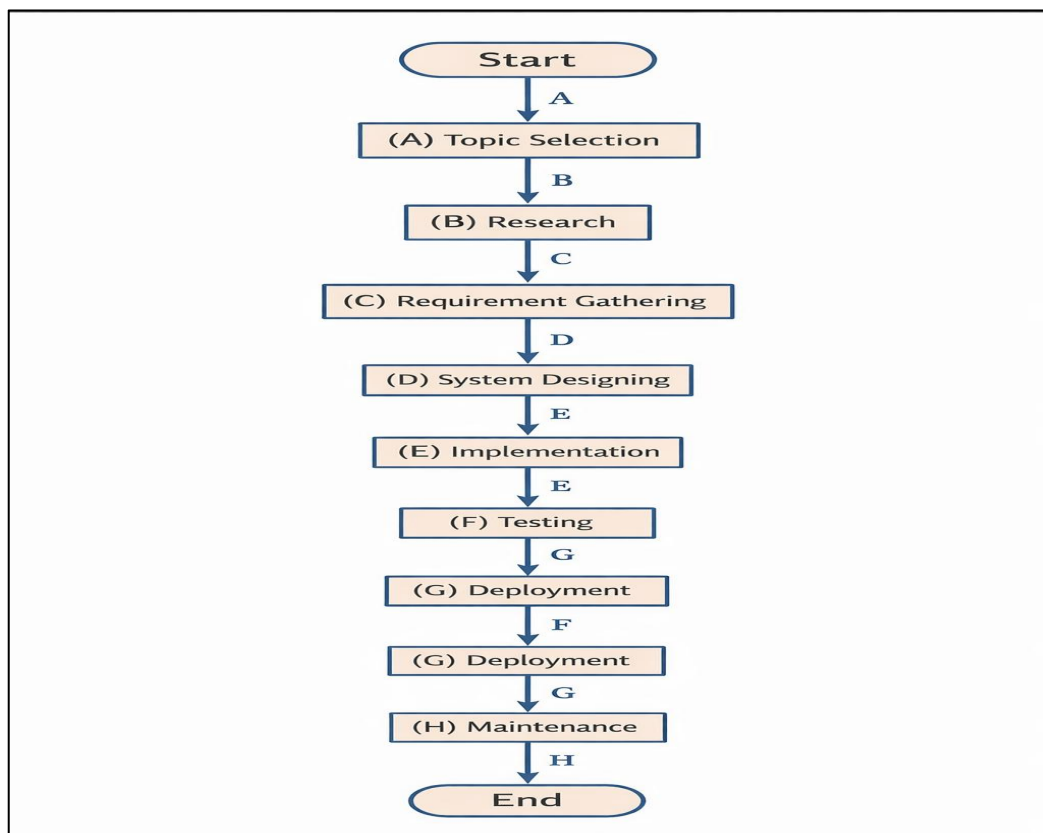


Figure 3.3.3 PERT Diagram

3.4 Software and Hardware Requirements:

3.4.1 Software Requirements:

- **Operating System:** Windows 10 / Linux / macOS
- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js with Express.js
- **Database:** MongoDB
- **IDE / Editor:** Visual Studio Code
- **Browser:** Chrome, Firefox, or Edge (latest versions)

3.4.2 Hardware Requirements:

For Development Environment (Admin/Developer):

- a. Processor: Intel i5 / AMD equivalent or higher
- b. RAM: 8 GB or above
- c. Hard Disk: Minimum 30 GB
- d. Network: Stable internet connection.

3.5 Preliminary Product Design

The preliminary product design provides an overview of the Stationery Store Web Application in terms of its layout, architecture, database schema, and key modules. This design phase ensures that the system meets both functional and non-functional requirements before detailed development begins.

1. System Architecture

The system follows a three-tier architecture:

I. Presentation Layer (Frontend):

Developed using HTML, CSS, and JavaScript. This includes pages such as Home Products, Cart, Login, Register, About, Contact, and Orders.

II. Application Layer (Backend):

Built with Node.js and Express.js. It manages API calls, authentication, product handling, order processing, and payment validation.

III. Data Layer (Database):

Uses MongoDB for storing user details, product catalogs, and order records.

2. User Interface Design

The UI is designed to be **simple, intuitive, and responsive**.

Key screens include:

- **Home Page:** Highlights stationery categories and promotions.
- **Products Page:** Grid layout of products with Add to Cart buttons.
- **Cart & Checkout:** Displays selected items, quantities, and total price.
- **Payment Screen:** Options for Cash on Delivery and Card Payment.
- **Orders Page:** Lists past purchases with status.
- **Static Pages:** About Us and Contact for store information.

3. Database Design

- **User Collection:** Stores user accounts (username, password).
- **Product Collection:** Stores product details (name, description, price, image).
- **Order Collection:** Stores orders with (userId, items, paymentMethod, createdAt)

2. Preliminary Module Design

- **Authentication Module:** Register, Login, Logout.
- **Product Management Module:** Add, Update, Delete, View (Admin).
- **Printing Module :** Upload files for Print .
- **Cart & Checkout Module:** Add to cart, update quantity, confirm order.
- **Payment Module:** COD and Card options.
- **Order Management Module:** Track past and current orders.

3. Design Constraints & Considerations

- The system assumes local MongoDB connection.
- Payment is mocked (no real gateway yet).
- Initial version focuses on basic UI/UX, future versions may adopt Bootstrap or Tailwind for responsiveness.

3.6 Conceptual Models

3.6.1 Entity -Relationship Diagram

E-R (Entity-Relationship) Diagram is used to represents the relationship between entities in a table. ER diagrams represent the logical structure of databases. ER Diagram relationship two database tables. E-R diagram means Entity Relationship diagram. Entity is an object of system, generally we refer entity as database table , the e-r diagram represents the relationship between each table of database. E- R diagram represent entity with attributes is a properties of entity. If we assume entity is a database table then all the columns of table are treated as attributes

The Entity Relationship (ER) Diagram represents the logical structure of the database for the stationery store system. It identifies the main entities, their attributes, and the relationships among them.

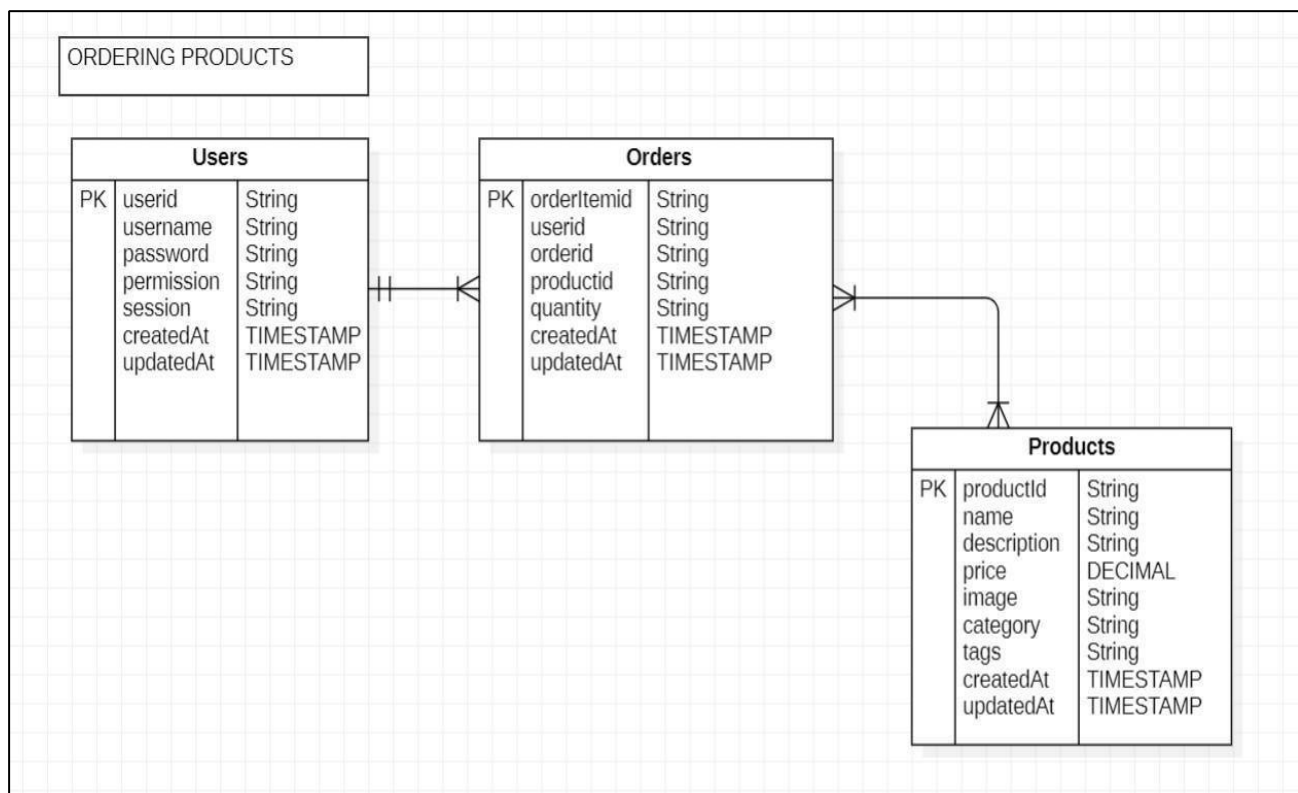


Figure 3.6.1: E-R Model of Project

3.6.2 Use Case Diagram

Use Case Diagram A use case diagram is used to represent the dynamic behavior of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles a system

1. The User focuses on buying stationery (shopping flow).
2. The Admin focuses on managing the system (products, users, orders).
3. <<include>> means the use case is always part of another action.
4. <<extend>> means the use case is optional or conditional

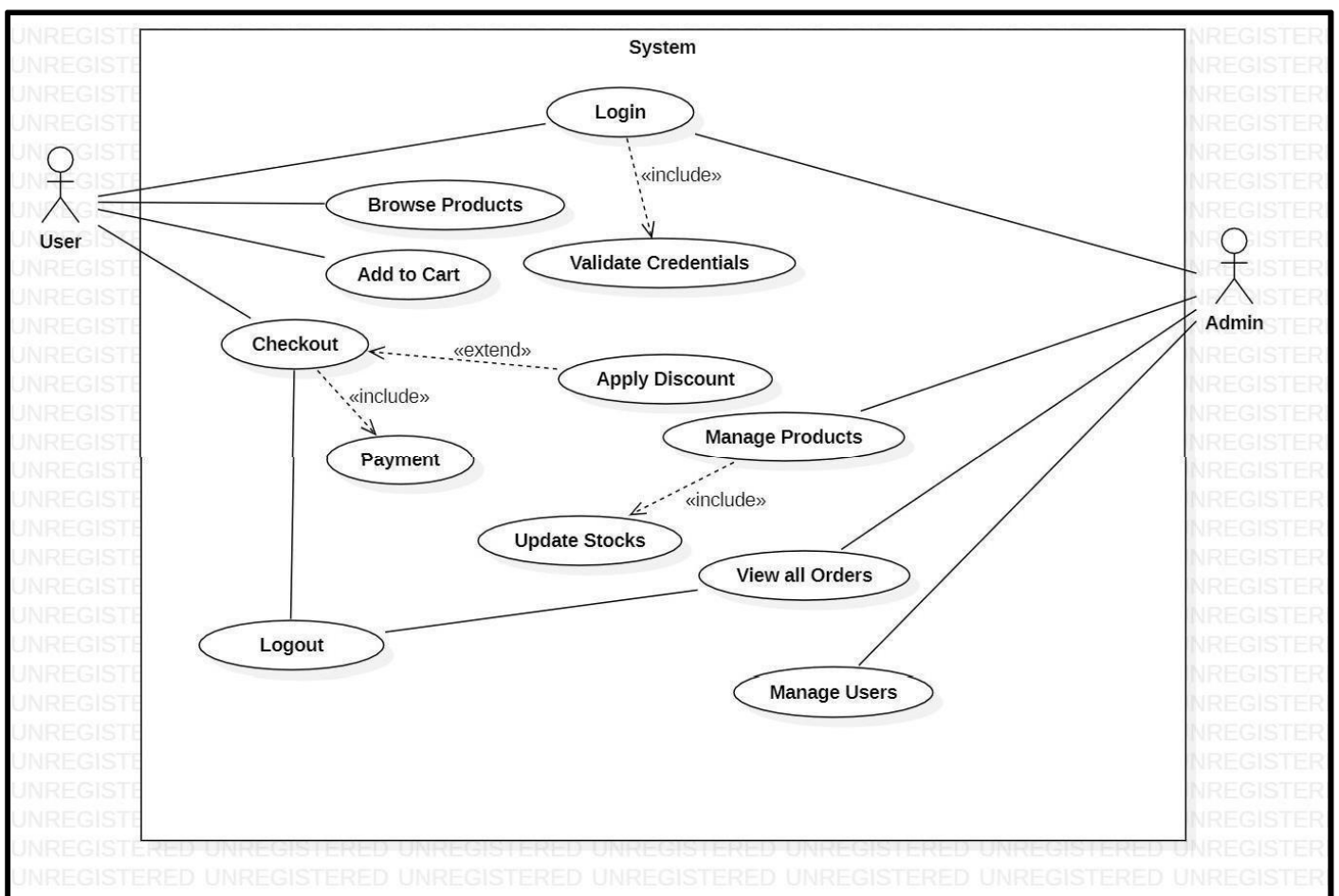


Figure 3.6.2 Use Case Diagram

3.6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation that shows how data moves through a system, the processes that transform the data, and how it interacts with external entities.

- **External Entities** → Sources or destinations of data (e.g., user, system).
- **Processes** → Actions that transform data (e.g., calculation, validation).
- **Data Stores** → Where data is stored (e.g., database, file).

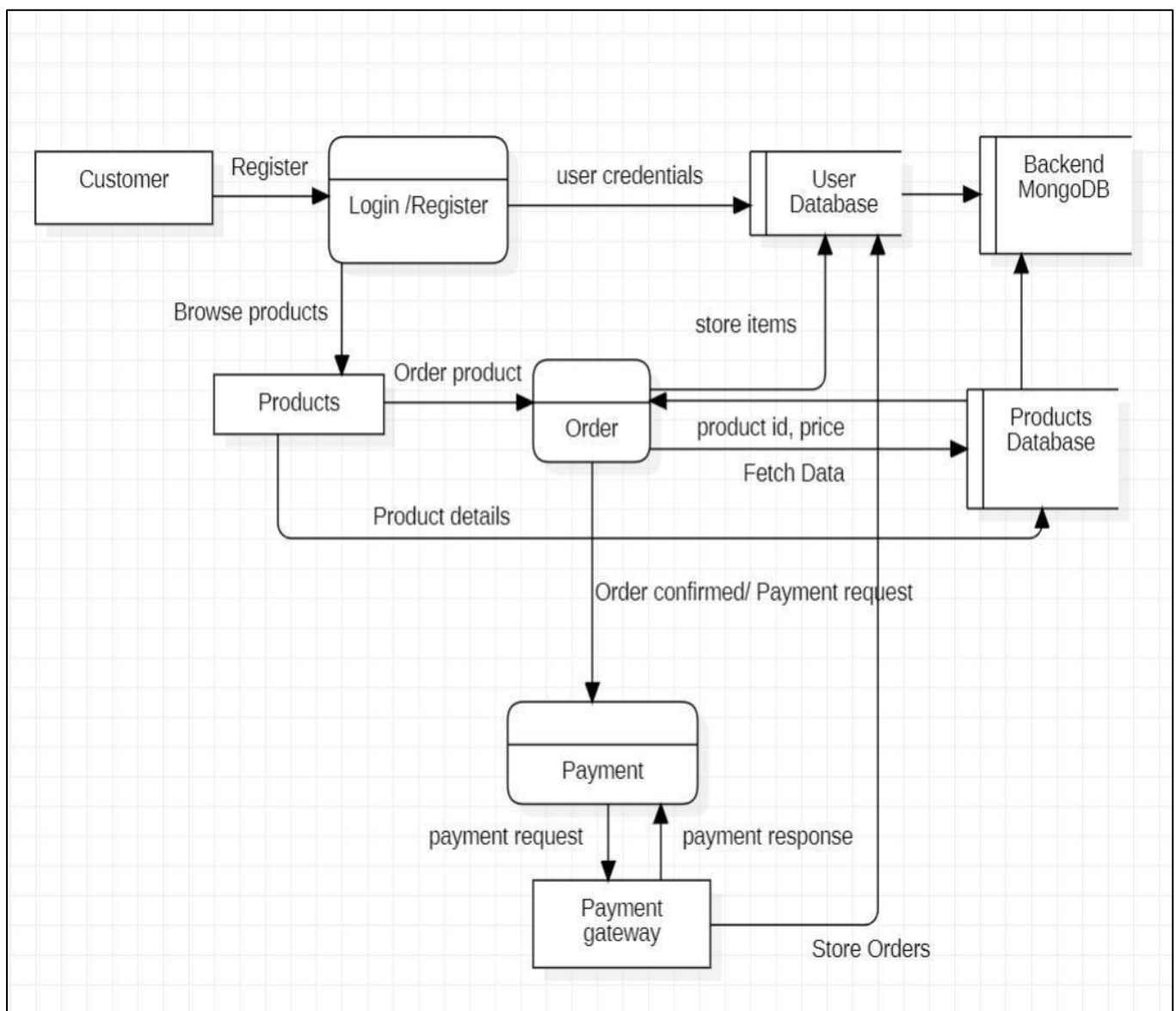


Figure 3.6.3 DataFlow Diagram

3.6.4 Sequence Diagram

The sequence diagram represents the flow of messages in the system and is also termed as an event diagram. It helps in envisioning several dynamic scenarios. It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelines took part at the run time. In UML, the lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page. It incorporates the iterations as well as branching

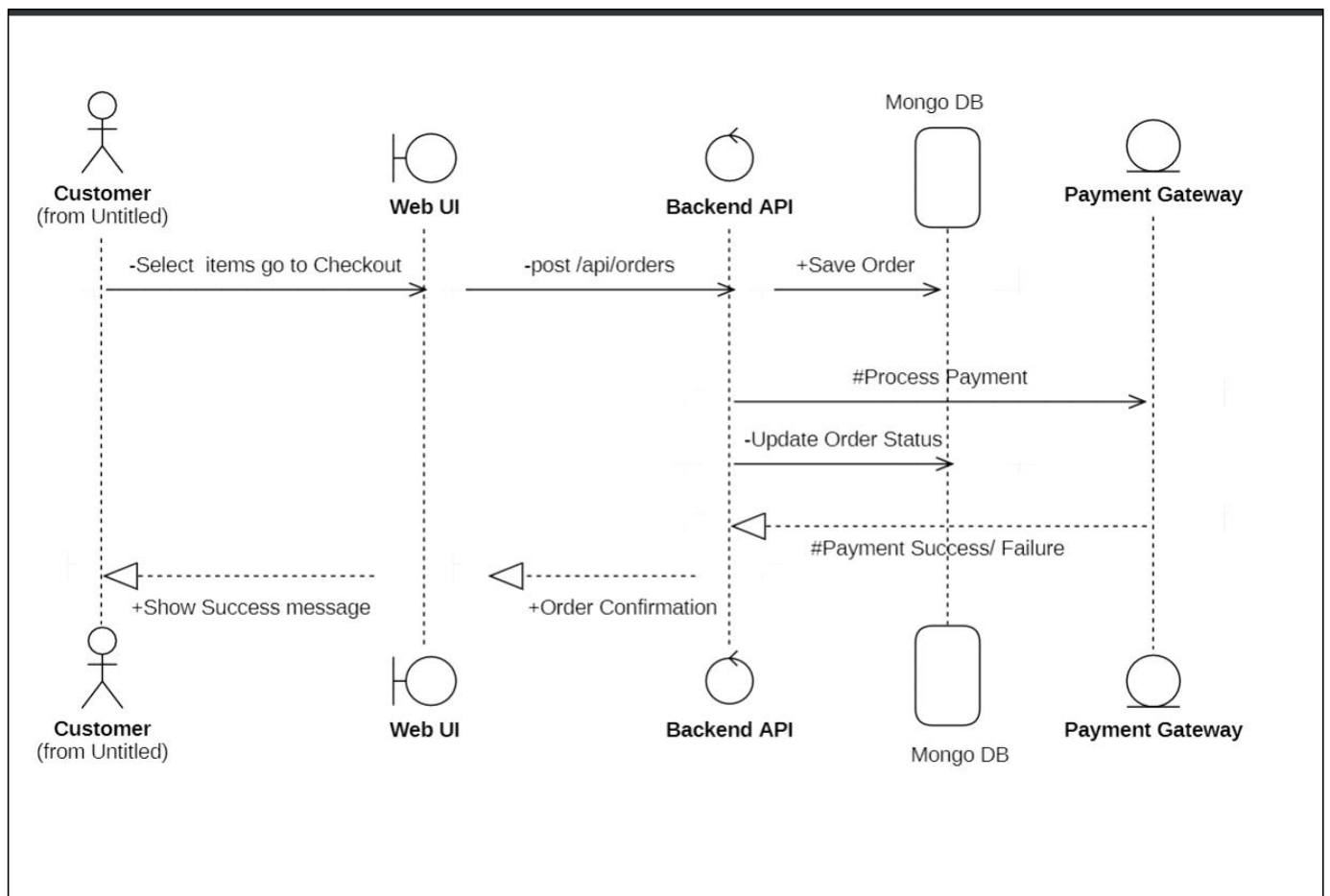


Figure 3.6.4 Sequence Diagram

CHAPTER 4 : SYSTEM DESIGN

4.1 Basic Modules

User Management Module

This module manages all user-related functionalities. New users can register with their details, while existing users can log in securely. It also handles authentication (session/cookie-based), logout, and provides profile management features such as updating user details or changing passwords. This ensures only authenticated users can place orders or view order history.

Product Management Module

The product management system allows users to browse stationery items, filter them by categories, and search for specific products. Each product has a detailed page showing name, price, description, and availability. Admins can add new products, edit existing ones, or remove outdated items from the system.

Shopping Cart Module

Users can add products to their cart, update quantities, or remove items. The cart displays a summary of selected products, prices, and total cost before checkout. It provides a convenient way for users to manage what they want to buy before placing the final order.

Order Management Module

This module handles the complete order process. Once the cart is finalized, the user can place an order by selecting a payment method (Cash on Delivery, Card, UPI). The system then records the order, generates a unique order ID, and stores the status (Pending, Shipped, Delivered, etc.). Users can also view their order history and check the status of past purchase

Shipping & Address Module

Users are required to enter their shipping address (full address, city, pincode) when placing an order. The address is linked to their order and stored in the database for future reference. With Google Maps API integration, the system can generate navigation links for delivery tracking, improving the shipping experience

Payment Module

This module integrates with payment gateways for secure transactions. Users can choose between multiple payment options like Cash on Delivery (COD), Card Payments, or UPI. The system verifies payment success or failure and updates the order status accordingly. COD orders are marked as pending until delivery.

Admin Module

The admin dashboard provides control over the entire system. Admins can manage users, add or edit products, view all orders placed, and update order statuses (e.g., from Pending → Shipped → Delivered). This ensures smooth business operations and order fulfillment.

4.2 Data Design

The stationery shop project uses MongoDB as the database to store and manage all information related to users, products, orders, payments, and shipping. The database is designed to support efficient e-commerce operations and ensure smooth interaction between different modules.

4.2.1 Schema Design

1. User Collection

Field Name	Data Type	Constraints	Description
user_id	INT	PRIMARYKEY, AUTO_INCREMENT, UNIQUE	Unique ID for each user.
username	STRING	NOT NULL, UNIQUE	User's login name.
email	STRING	NOT NULL, UNIQUE	User's email address.
password	STRING	NOT NULL	Hashed password.
role	STRING	DEFAULT 'customer'	User role (e.g., customer, admin).
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation time.

4.2.1 User Schema

2. Product Collection

Field Name	Data Type	Constraints	Description
product_id	INT	PRIMARYKEY, AUTO_INCREMENT, UNIQUE	Unique product ID.
name	STRING	NOT NULL	Product name.
description	TEXT	NULL	Product description.
category	STRING	NOT NULL	Product category (e.g., pens, paper).
price	DECIMAL	NOT NULL	Product price.
stock_quantity	INT	NOT NULL	Available stock count.
image_url	BLOB/STRING	NULL	Product image path.

4.2.2 Products Schema

3. Cart Collection

Field Name	Data Type	Constraints	Description
cart_id	INT	PRIMARYKEY, AUTO_INCREMENT, UNIQUE	Unique cart ID.
user_id	INT	FOREIGN KEY → users(user_id)	Customer who owns the cart.
product_id	INT	FOREIGN KEY → products(product_id)	Product added to cart.
quantity	INT	NOT NULL	Quantity of product in cart.
added_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the product was added to cart.

4.2.3 Cart Schema

4. Order Collection

Field Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT, UNIQUE	Unique product ID.
name	STRING	NOT NULL	Product name.
description	TEXT	NULL	Product description.
category	STRING	NOT NULL	Product category (e.g., pens, paper).
price	DECIMAL	NOT NULL	Product price.
stock_quantity	INT	NOT NULL	Available stock count.
image_url	STRING	NULL	Product image path.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date added.

4.2.4 Order Schema

5. Payment Collection

Field Name	Data Type	Constraints	Description
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT, UNIQUE	Unique ID for each payment.
order_id	INT	FOREIGN KEY → orders(order_id)	Linked order for this payment.
user_id	INT	FOREIGN KEY → users(user_id)	Customer making the payment.
amount	DECIMAL	NOT NULL	Payment amount.
payment_method	STRING	NOT NULL	Method (e.g., UPI, Card, NetBanking, COD).
payment_status	STRING	DEFAULT 'Pending'	Status (e.g., Pending, Success, Failed).
transaction_id	STRING	UNIQUE, NULL allowed	Gateway transaction/reference ID.

4.2.5 Payment Schema

4.2.2 Data Integrity And Constraints

Data integrity ensures that the information stored in the database is accurate, consistent, and reliable. Constraints are applied to maintain logical correctness and prevent invalid data from entering the system.

1 . Entity Integrity

Each collection (User, Product, Order, Payment, Shipping) has a unique identifier (`_id` in MongoDB, auto-generated).

No two records can have the same identifier

Example: Every User has a unique `userId`, and every Order has a unique `orderId`.

2 . Referential Integrity

Relationships between collections are maintained using references (`ObjectId`).

Orders always reference a valid `userId` and `productId`.

Payments always reference a valid `orderId`.

Shipping records always reference a valid `orderId`.

Example: An order cannot exist without a valid user, and a payment cannot exist without a valid order

3. Domain Integrity

Attributes must take only valid values.

Examples:

Payment Method → Allowed values: "Cash on Delivery", "Card".

Order Status → Allowed values: "Pending", "Shipped", "Delivered", "Failed".

Pincode must be numeric and fixed length (e.g., 6 digits).

Email must be unique and follow a valid format.

4. User Constraints

Username and email must be unique across the system.

Password must be stored in encrypted/hashed form for security.

Role must be either "Customer" or "Admin".

5. Product Constraints

Product price must always be a positive number.

Product stock cannot be negative.

Product name must be unique within the same category

6. Order Constraints

Orders must contain at least one item.

quantity of each product in the order must be at least **1**.

An order cannot be placed if cart is empty.

7. Payment Constraints

A payment must always reference a valid order.

amount must equal the total order value

status must be one of "Success", "Pending", or "Failed".

4.3 Procedural Diagram

Logical Diagram

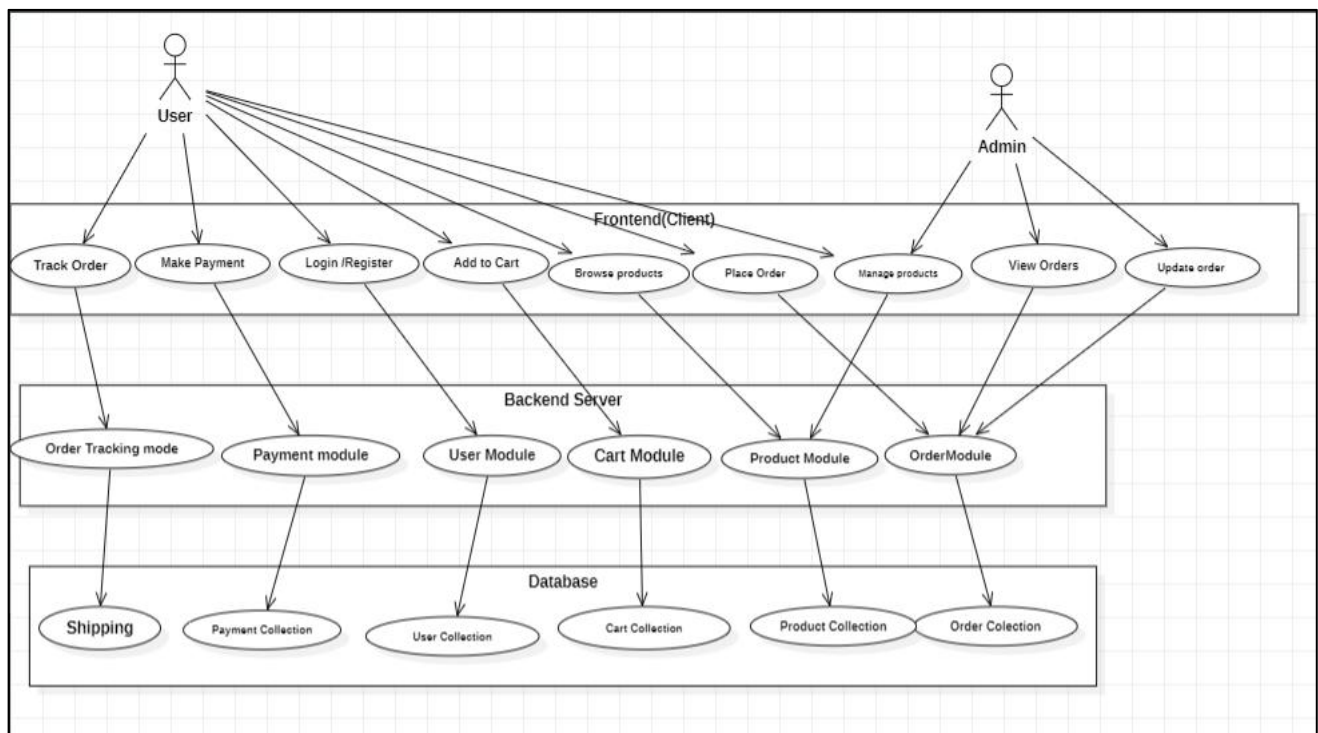


Figure 4.3.1 Logical Diagram of Project

4.3.1 Data Structures

1. Arrays / Lists

Used in Cart (items[]) and Orders (order.items[]).

Example: cart = [{ productId, name, price, qty }, ...]

· 2. Hash Maps / Objects

Used for quick lookup of users, products, and sessions.

Example: { email: userObject }

· 3. Stack (Implicit in session management)

User login/logout works like push/pop of sessions.

· 4. Queue (Order Processing)

Orders are placed in sequence and processed FIFO (First In, First Out).

· 5. Database Collections (MongoDB documents)

Abstract DS representing persistent storage for Users, Products, Orders, Payments, Shipping.

4.3.2 Algorithm

1 . Searching

Linear Search → Browsing products by keyword.

Indexed Search (MongoDB) → Find product by id, name, category.

2. Sorting

Sort products by price, name, popularity.

Sort orders by date or status.

3. Insertion & Deletion

Insert new user/product/order.

Delete from cart or database.

4.Hashing

Passwords are hashed for secure authentication\ delivery routes.

5.Validation Algorithms

Check valid email format.

Validate pincode length.

Program Flow Algorithm (Start → Stop)

Algorithm: Stationery Shop System

1. Start

2. User Access

Display homepage (products, login/register links).

If new user → Register account.

If existing user → Login (validate credentials).

3. Browse Product

Fetch product list from database.

Display products with search/sort options.

Printing Options for upload documents

4. Add to Cart

User selects products & quantity.

Store items in cart (localStorage / DB).

5. Proceed to Checkout

Validate cart (not empty, stock available).

Redirect to Shipping Form.

6. Enter Shipping Detail

User enters address, city, pincode

Validate inputs (correct format).

7. Select Payment Method

Options: Cash on Delivery / Card .

Confirm order total

Download invoice.

8. Place Order

Save order in database with status = “Pending”.

Generate unique Order ID.

9. Payment Processing

If COD → Mark payment “Pending”.

If Card → Process payment → Update status.

10. Shipping & Tracking

Store shipping details.

Allow user to track order (status + Google Maps navigation demo).

11. Admin Operations (parallel)

Admin manages products (add/update/delete).

Admin views orders and updates order status.

12. Order Completion

Once delivered, update status to “Delivered”.

Notify user.

13. Logout / Exit

End user session.

14. Stop

4.4 User Interface Design

4.4.1 Home Page

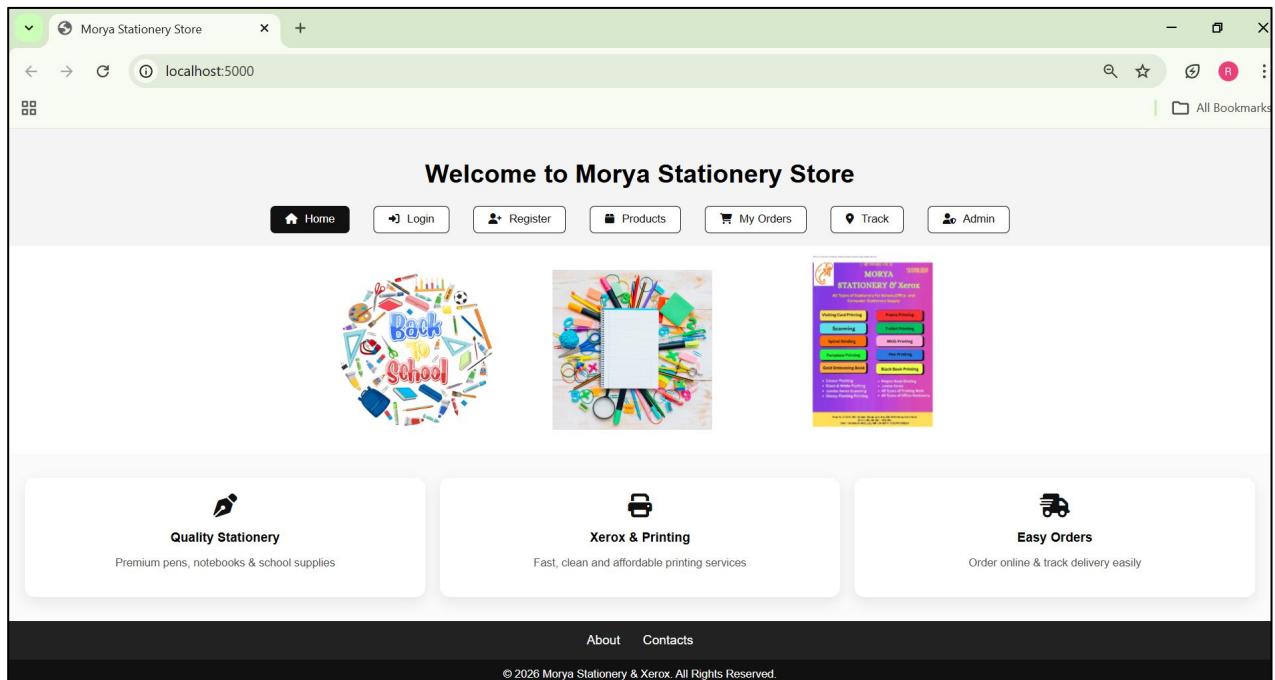


Figure 4.4.1 Home Page

The home page acts as the entry point of the stationery shop system. It provides an overview of the store, highlights available products, and allows users to navigate to different sections such as products, cart, orders, login, and contact. It must be user-friendly, attractive, and responsive to engage customers.

4.4.2 Login Page

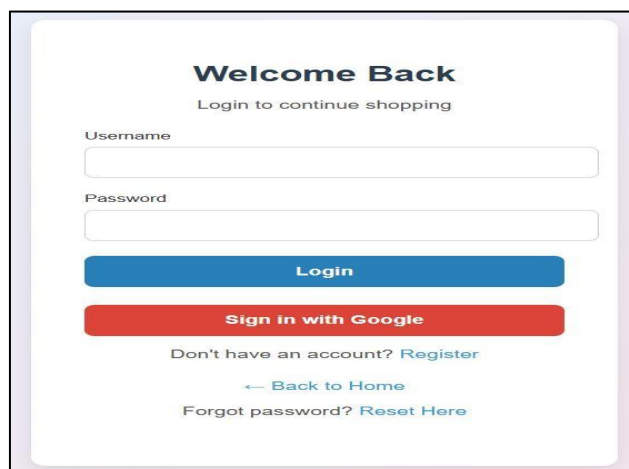
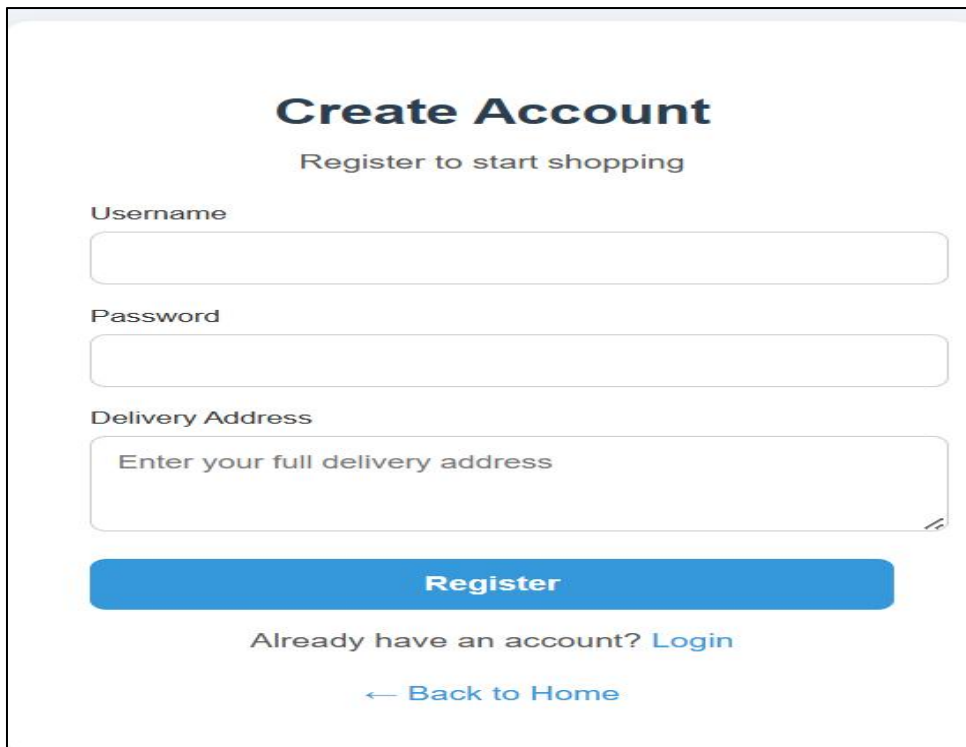


Figure 4.4.2 Login Page

A "User Login Page" is a specific web page within a system or application where users can enter their credentials, such as a email and password, to gain access to their accounts. It typically includes input fields for users to enter their login information.

4.4.3 Register Page



Create Account

Register to start shopping

Username

Password

Delivery Address

Enter your full delivery address

Register

Already have an account? [Login](#)

[← Back to Home](#)

Figure 4.4.3 Register Page

The Register Page allows new users to create an account by entering basic details such as name, email, password, and address. The system validates user input and stores the data securely in database. This enables users to log in, place orders, and access personalized features of the website.

4.4.4 Products Page

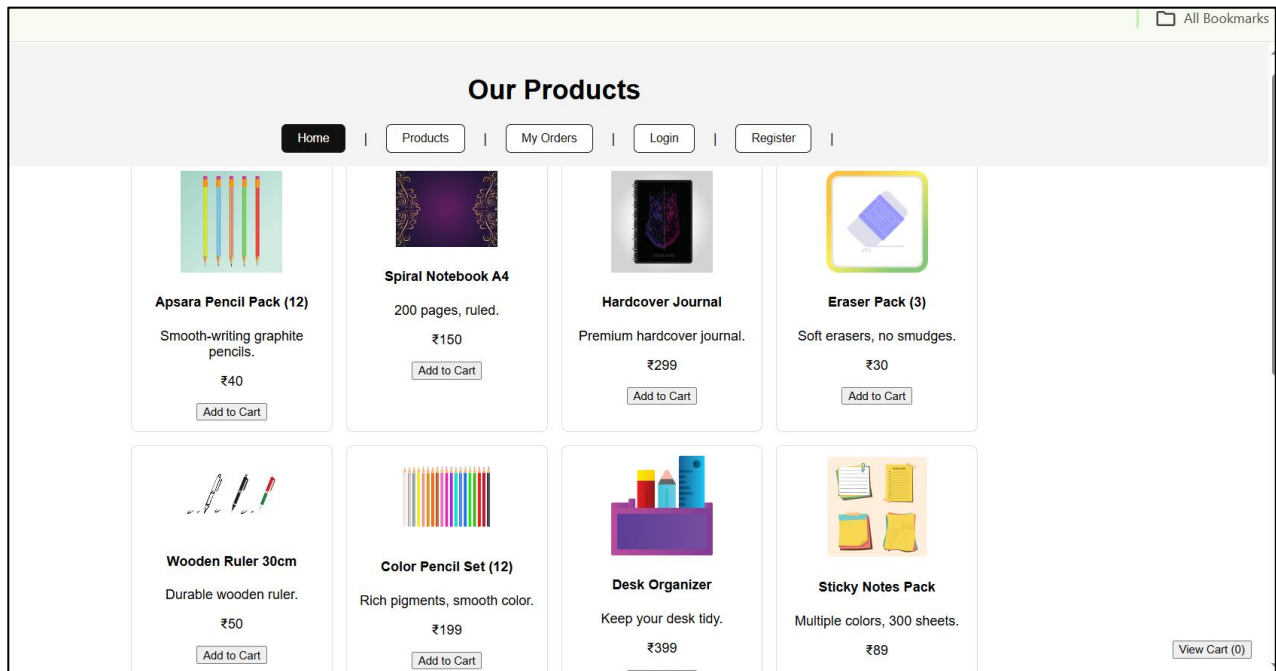


Figure 4.4.4 Products Page

The Product Page displays all available stationery items with details such as product name, price, image, and description. Users can browse products, view details, and add items to their cart. The page provides a user-friendly interface to help customers easily select and purchase products.

4.5 Security Issues

1. Weak Authentication and Password Management

Issue:

Passwords stored in plain text or using weak hashing algorithms (like MD5 or SHA1) can be easily cracked, allowing attackers to access user accounts.

Solution:

Use strong password hashing algorithms like bcrypt, Argon2, or PBKDF2.

Enforce strong password policies (minimum length, special characters, etc.).

Implement multi-factor authentication (MFA) for sensitive actions.

2. SQL/NoSQL Injection

Issue:

Unsanitized user inputs in login or order forms can allow attackers to inject malicious queries and gain unauthorized database access.

Solution:

Use parameterized queries or prepared statements.

Validate and sanitize all user inputs.

Use an Object Relational Mapper (ORM) to interact with the database safely

3. Cross-Site Scripting (XSS)

Issue:

If user input is displayed without escaping, attackers can inject JavaScript code to steal cookies or manipulate site content.

Solution:

Escape all user inputs before displaying them.

Use built-in XSS protection frameworks (like Angular's sanitization or React's JSX escaping).

Implement a Content Security Policy (CSP) to block unauthorized scripts

4. Cross-Site Request Forgery (CSRF)

Issue:

Attackers can trick logged-in users into performing unwanted actions on the website.

Solution:

Include CSRF tokens in all form submissions.

Verify Referer or Origin headers for sensitive operations

Use SameSite cookies to prevent unauthorized cross-site requests.

5. Improper Session Management

Issue:

If session tokens are not secured or expired properly, attackers can hijack active sessions.

Solution:

Use secure and HTTP-only cookies.

Implement automatic session timeout and logout after inactivity.

Regenerate session IDs after login and logout.

6. Insecure Data Handling and Transmission

Issue:

Sensitive information like passwords, addresses, and payment data may be stolen if transmitted or stored insecurely.

Solution:

Use HTTPS (SSL/TLS) for all communications.

Encrypt sensitive data before storing it.

Do not store unnecessary payment information on the server

4.6 Test Cases

1.Registration Page

Test ID	Test Scenario	Precondition	Test Steps	Expected Result
TC-01	User Registration Login	User not registered	Open Signup page Enter valid details, Address Submit form	Account created successfully, user redirected to login page
TC-02	Duplicate Registration	Email already registered	Try signing up with an existing email	System shows "Email already exists" error

Table 4.6.1 Registration test case

2.Login Page

TC-03	User Login Success	Valid credentials exist	Open Login page Enter correct username/email & password Submit	User successfully logs in and redirected to homepage
TC-04	User Login Failure	Invalid credentials	1. Enter wrong password 2. Click login	System displays "Invalid username/password"

Table 4.6.2 Login test case

3.Products Order

TC-05	View Products	Products exist in database	1. Navigate to Products page	All available products displayed with name, price, and image
TC-06	Search Product	Products exist in database	Upload files in printing	Only relevant products displayed
TC-07	Add Item to Cart	User logged in	1. Select product 2. Click“Add to Cart”	Item appears in cart, cart count increases
TC-08	Update Cart Quantity	Product already in cart	1. Open cart 2. Change quantity 3. Update cart	Total price updates correctly
TC-09	Remove Item from Cart	Product already in cart	1. Open cart 2. Remove item	Item removed, total recalculated
TC-10	Place Order (COD) /Card	User logged in, Cart filled	1. Openshipping form 2. Fetch address 3. SelectCOD/ Card and Submit	Order created successfully with status = “Pending” Download Invoice

Table 4.6.3 Products order test case

4.Logout

Test ID	Test Scenario	Precondition	Test Steps
TC-03	User Login Success	Valid credentials exist	1. Open Login page 2. Enter correct credentials 3. Submit
TC-04	User Login Failure	Invalid credentials	1. Enter wrong Password 2. Click login

Table 4.6.4 Logout test case

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Approaches:

The Stationery E-Commerce System is implemented using a client–server architecture. The application is divided into three major layers: Frontend, Backend, and Database. The frontend is developed using HTML, CSS, and JavaScript, providing a user-friendly interface for customers to browse products, register, login, and place orders.

The backend is developed using Node.js and Express.js, which handles business logic, user authentication, product management, and order processing. MongoDB is used as the database to store user details, product, and order information. The system follows a modular implementation approach, where each module such as user authentication, product management, order handling, and admin management is implemented independently and later integrated.

The system follows a modular implementation approach, where each module such as user authentication, product management, order handling, and admin management is implemented independently and later integrated.

5.1.1 AGILE MODEL APPROACH

The Agile Software Development Model is an iterative and incremental approach to software development in which the project is divided into small development cycles called iterations or sprints. Each iteration results in a working version of the software, which is continuously improved based on feedback.

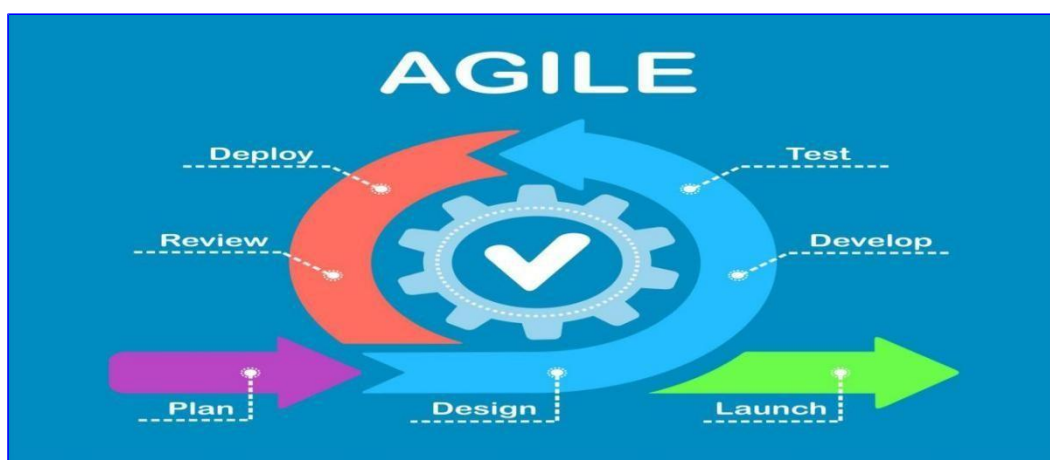


Figure 5.1.1 Agile Model

Why Agile Model was Used in This Project

The Stationery E-Commerce Website required frequent changes in:

- ❖ User interface design
- ❖ Product management features
- ❖ Admin functionalities
- ❖ Order handling workflow
- ❖ The Agile model supports continuous improvement and flexibility, making it suitable for this project.

Advantages of Agile Model

- ❖ Flexible to requirement changes
- ❖ Early delivery of working software
- ❖ Continuous testing and feedback
- ❖ Improved customer satisfaction
- ❖ Reduced project risk

Disadvantages of Agile Model

- ❖ Difficult to estimate time and cost
- ❖ Requires continuous user involvement

5.1.2 INCREMENTAL APPROACH

The Incremental Model is a software development approach in which the system is developed and delivered in small functional parts (increments). Each increment adds new features while using the previously developed system as a base.

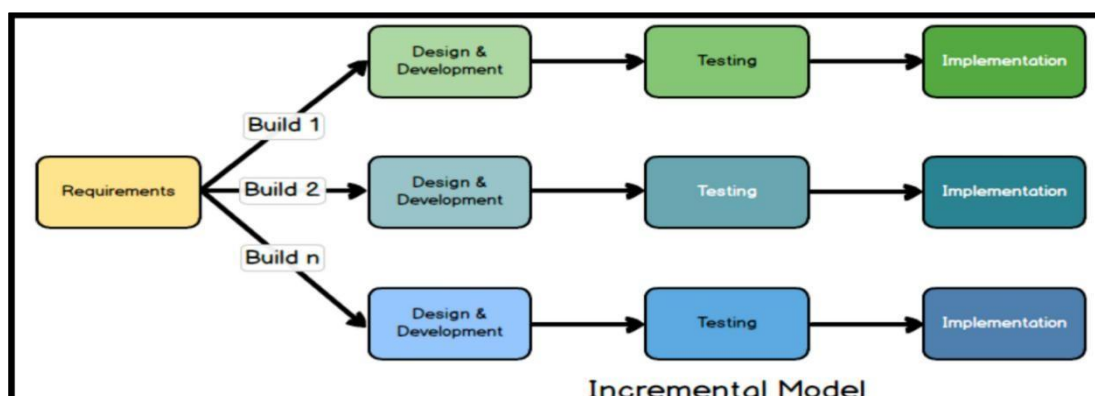


Figure 5.1.2 Incremental Model

Increment No.	Module Name	Functionalities Implemented
Increment 1	Basic System	Home page UI design, Navigation bar, Image slider, Static pages (About, Contact)
Increment 2	User Authentication	User registration (Email & Mobile), Login functionality, Session handling, MongoDB user data storage
Increment 3	Product Module	Product listing, Product images display, Fetching products from MongoDB database, Add to cart functionality (Frontend), Upload files for printing.
Increment 4	Order Module	Order placement, Storing orders in database, View “My Orders” page, Order tracking using Order ID

5.1.2 Increment Model Approach Table

5.1.3 SCRUM MODEL APPROACH

The Scrum Model was effectively used in the Stationery E-Commerce project to deliver the system in multiple sprints. It ensured flexibility, continuous testing, and timely delivery of features such as product management, orders processing and admin control.

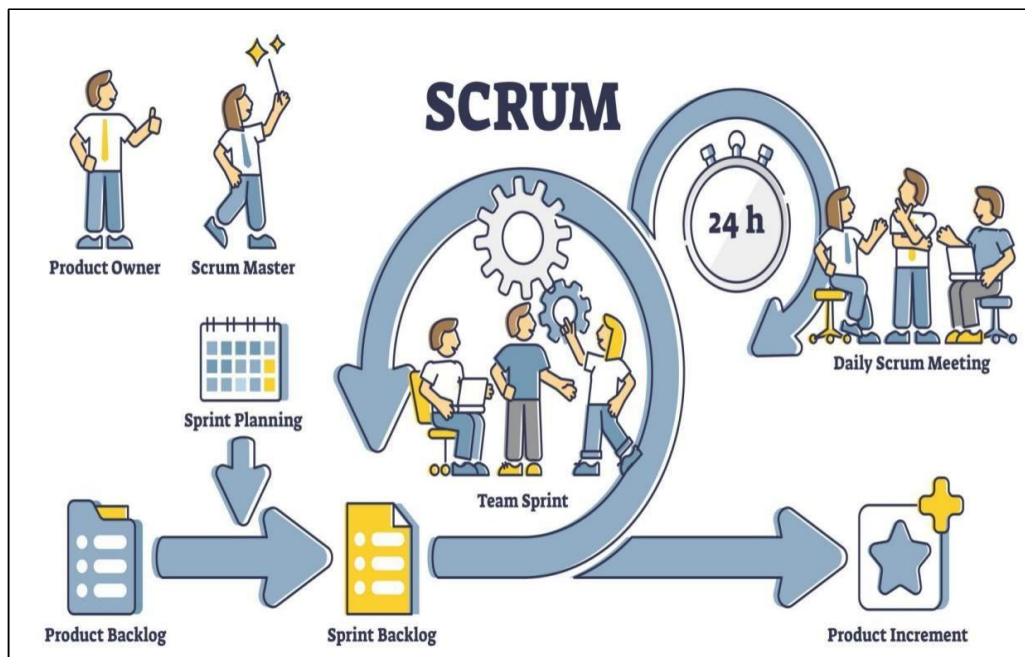


Figure 5.1.3 Scrum mod

Advantages of Scrum Model

- ❖ Fast delivery of working software
- ❖ Easy to handle changing requirements
- ❖ Continuous feedback and improvement

Disadvantages of Scrum Model

- ❖ Difficult to estimate total cost and time
- ❖ Not suitable for very small projects

5.2 Code Details and Efficiency

The backend code is written in JavaScript using the Express framework. RESTful APIs are created for different functionalities such as registration, login, product listing, order placement, and order tracking.

Key code features include:

- Use of Mongoose schemas for structured data storage
- Session-based authentication for user login
- Separation of routes for users and admin
- Use of reusable components for frontend pages
- Efficient database queries using MongoDB indexing

The system ensures efficient performance by minimizing redundant database calls and using asynchronous programming for faster response times.

5.2.1 Home Page

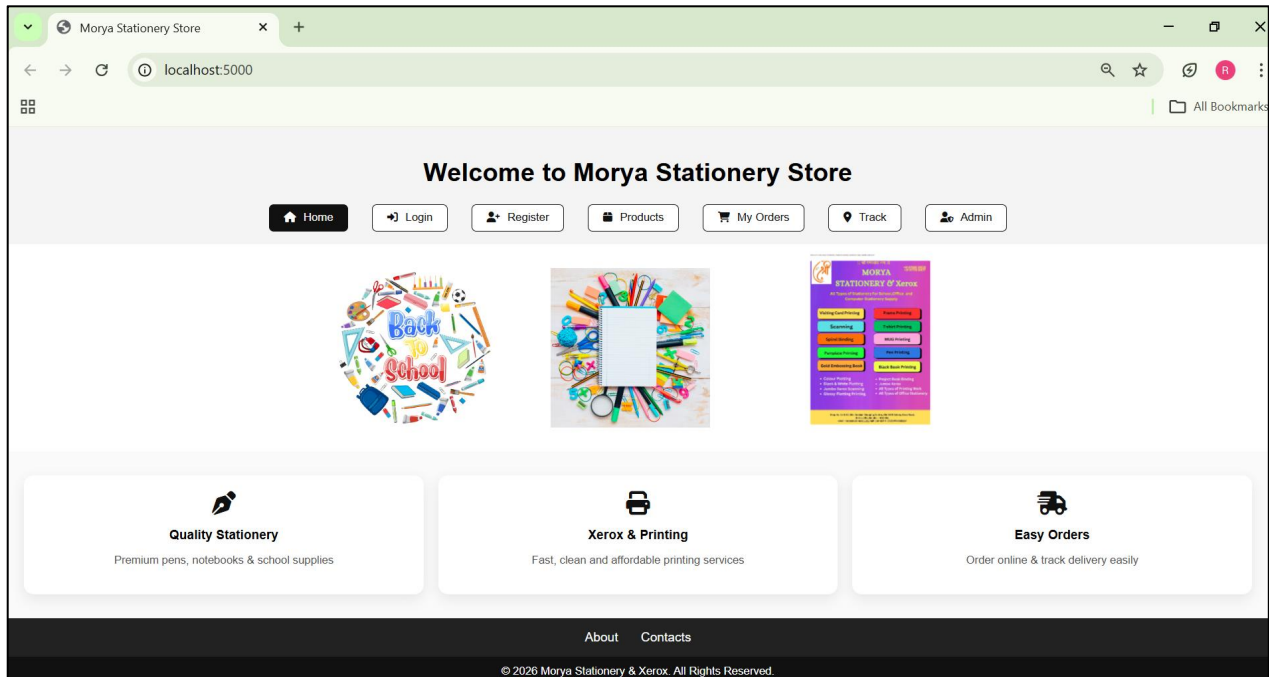


Figure 5.2.1 Home Page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Morya Stationery Store</title>
<link rel="stylesheet" href="styles.css">
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
</head>
<body>
<header>
<h1>Welcome to Morya Stationery Store</h1>
<nav class="top-nav">
<a class="nav-btn" href="index.html"><i class="fa fa-house"></i> Home</a>
<a class="nav-btn" href="login.html"><i class="fa fa-right-to-bracket"></i> Login</a>
<a class="nav-btn" href="register.html"><i class="fa fa-user-plus"></i> Register</a>
<a class="nav-btn" href="products.html"><i class="fa fa-box"></i> Products</a>
<a class="nav-btn" href="order.html"><i class="fa fa-cart-shopping"></i> My Orders</a>
<a class="nav-btn" href="order-tracking.html"><i class="fa fa-location-dot"></i> Track</a>
<a class="nav-btn" href="admin-login.html"><i class="fa fa-user-shield"></i> Admin</a>
</nav>
</header> SCREEN SLIDER
<section class="hero">
```



```
<div class="slider">
<div class="slides">



</div>
</div>
```

```
</section> CARD SECTION
```

```
<section class="card-section">
<div class="card">
<i class="fa fa-pen-nib"></i>
<h3>Quality Stationery</h3>
<p>Premium pens, notebooks & school supplies</p>
</div>
<div class="card">
<i class="fa fa-print"></i>
<h3>Xerox & Printing</h3>
<p>Fast, clean and affordable printing services</p>
</div>
```

```
<div class="card">
<i class="fa fa-truck-fast"></i>
<h3>Easy Orders</h3>
<p>Order online & track delivery easily</p>
</div>
</section>
```

```
FOOTER
```

```
<footer>
<div class="footer-links">
<nav>
<a href="about.html">About</a>
<a href="contact.html">Contacts</a>
<a href="email.html">Register Email</a>
<a href="mobile.html">Mobile Login</a>
</nav>
</div>
```

```
<div class="copyright">
© 2026 Morya Stationery & Xerox. All Rights Reserved.
</div>
</footer>
<script src="script.js"></script>
</body>
</html>
```

5.2.2 LOGIN PAGE :

Welcome Back

Login to continue shopping

Username

Password

Login

Sign in with Google

Don't have an account? [Register](#)

[← Back to Home](#)

Forgot password? [Reset Here](#)

Figure 5.2.2 Login Page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Login - Stationery Store</title>
<style>body{ margin:0;
padding: 0;
font-family: "Poppins", sans-serif;
background: linear-gradient(135deg, #e3f2fd, #fce4ec); height: 100vh;
display: flex;
justify-content: center;
align-items: center; }
.login-container { background: #ffffff;
padding: 40px; width: 360px; box-
shadow: 0 8px 20px rgba(0,0,0,0.1);
border-radius: 12px;
```

```
    animation: fadeIn 0.8s ease-in-out;  }  
@keyframes fadeIn {  
    from { opacity: 0; transform: translateY(20px); }  
    to { opacity: 1; transform: translateY(0); }  
}
```

```
.login-container h1  
{ text-align: center;  
  margin-bottom: 10px;  
  color: #2c3e50;  
  font-size: 28px;  
}
```

```
.login-container p  
{ text-align: center;  
  margin-bottom: 25px;  
  color: #555;  
}
```

```
.input-group {  
  margin-bottom: 20px;  
}  
.input-group label  
{ display: block;
```

```
font-size: 14px; margin-  
bottom: 6px; color:#333;}
```

```
.input-group input  
{ width: 100%;  
  padding: 12px;  
  border-radius: 8px;  
  border: 1px solid #ccc;  
  transition: 0.3s;  
  font-size: 15px;  
  outline: none;}
```

```
.input-group input:focus { border-color: #3498db;
box-shadow: 0 0 5px rgba(52,152,219,0.5);
}
```

```
button { width: 100%;
padding: 12px; border: none; background: #3498db; color: white;
font-size: 16px; font-weight: bold; border-radius: 8px; cursor: pointer; transition: 0.3s;
}
```

```
button:hover {
background: #2980b9;
}
```

```
margin-top: 12px;
background: #db4437;
}
```

```
#googleLoginBtn:hover
{ background: #c23321;
}
```

```
#loginMsg
{ margin-top:
12px; text-align:
center; color: red;
font-weight: 500;
}
```

```
#googleMsg
{ margin-top:
12px; text-align:
center; color:
green;
font-weight: 500;
}
```

```

.links {
  text-align: center;
  margin-top: 15px;
}

.links a {
  text-decoration: none;
  color: #3498db;
  font-weight: 500;
}

.links a:hover {
text-decoration: underline;
}
</style>
</head>
<body>
<div class="login-container">
<h1>Welcome Back</h1>
<p>Login to continue shopping</p>
<form id="loginForm">
<div class="input-group">
<label>Username</label>
<input type="text" id="username" required>
</div>
<div class="input-group">
<label>Password</label>
<input type="password" id="password" required>
</div>
<button type="submit">Login</button>
<div id="loginMsg"></div>
</form>

```

```
text-decoration: underline;
}
</style>
</head>
<body>
<div class="login-container">
<h1>Welcome Back</h1>
<p>Login to continue shopping</p>
<form id="loginForm">
<div class="input-group">
<label>Username</label>
<input type="text" id="username" required>
</div>
<div class="input-group">
<label>Password</label>
<input type="password" id="password" required>
</div>
<button type="submit">Login</button>
<div id="loginMsg"></div>
</form>
<button type="button" id="googleLoginBtn">Sign in with
Google</button>
<div id="googleMsg"></div>
<div class="links">
<p>Don't have an account? <a href="register.html">Register</a></p>
<a href="index.html">← Back to Home</a>
</div>
<div class="links">
<p>Forgot password? <a href="forgot.html">Reset Here</a></p>
</div>
```

```

<script>
document.getElementById('loginForm').addEventListener('submit', async (e)=>{ e.preventDefault();
const username = document.getElementById('username').value.trim();
const password = document.getElementById('password').value.trim(); try{
const res = await fetch('/api/login', { method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ username, password }), credentials: 'include'
});
const data = await res.json(); if(res.ok){
window.location.href = 'products.html';
} else {
document.getElementById('loginMsg').textContent = data.message || 'Login failed';
}
} catch(err){ console.error(err); }
});
</script>

<script type="module">
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import { getAuth, GoogleAuthProvider, signInWithPopup }
from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";
const firebaseConfig = {
apiKey: "AIzaSyBxas_UoXLbn0tu6NvQNRQoBj9FWFT4-uI",
authDomain: "stationery-website-c7cee.firebaseio.com",
projectId: "stationery-website-c7cee",
storageBucket: "stationery-website-c7cee.appspot.com",
messagingSenderId: "553718261048",
appId: "1:553718261048:web:cd2cc5804e5c649fe8cdcd"};

const app = initializeApp(firebaseConfig);

```

```

const auth = getAuth(app);
const googleBtn = document.getElementById("googleLoginBtn");
const googleMsg = document.getElementById("googleMsg");
if (googleBtn) { googleBtn.addEventListener("click", async () => { try {
googleMsg.innerText = "";
const provider = new GoogleAuthProvider();
const result = await signInWithPopup(auth, provider);
// create backend session
const res = await fetch("/api/firebase-login", { method: "POST",
headers: { "Content-Type": "application/json" }, credentials: "include",
body: JSON.stringify({ email: result.user.email })
});
const data = await res.json(); if (res.ok) {
googleMsg.innerText = "Google Login Successful! Redirecting..."; setTimeout(() => {
window.location.href = "products.html";
}, 1200);
} else {
googleMsg.innerText = "Backend login failed: " + data.message;
}
} catch (err) { console.error(err);
googleMsg.innerText = "Google login failed: " + err.code;
}
});
}
</script>
</body>
</html>

```


5.2.3 PRODUCT PAGE:

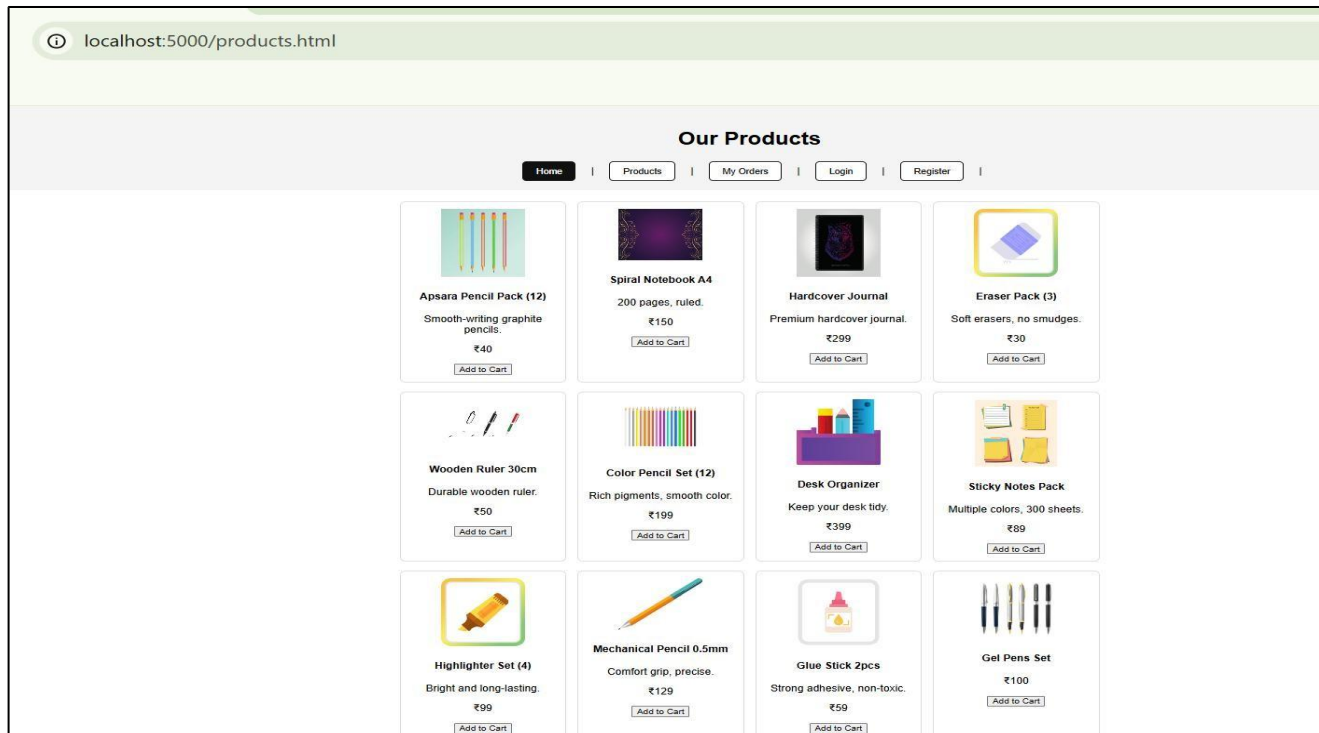


Figure 5.2.3 Product Page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Products - Morya Stationery Store</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
<header>
<h1>Our Products</h1>
<nav>
<a class="nav-btn" href="index.html">Home</a> |
<a class="nav-btn" href="products.html">Products</a> |
<a class="nav-btn" href="order.html">My Orders</a> |
<a class="nav-btn" href="login.html" id="loginLink">Login</a> |
<a class="nav-btn" href="register.html">Register</a> |
<a class="nav-btn" href="#" id="logoutLink" style="display:none;">Logout</a>
</nav>
</header>
```

```

<main id="productsGrid" class="products-grid"></main>
<div class="cart-bar">
<button id="viewCartBtn">View Cart (<span id="cartCount">0</span>)</button>
</div>
<div id="cartModal" class="modal" style="display:none;">
<div class="modal-content">
<h3>Your Cart</h3>
<div id="cartItems"></div>
<div id="cartTotal"></div>
<button id="checkoutBtn">Checkout (Place Order)</button>
<button id="closeCartBtn">Close</button>
</div> </div>
<div id="paymentModal" class="modal" style="display:none;">
<div class="modal-content">
<h3>Payment</h3>
<label>
<input type="radio" name="payment" value="COD" checked> Cash on Delivery
</label><br>
<label>
<input type="radio" name="payment" value="STRIPE"> Pay via UPI / Card
</label>
<button type="button"onclick="window.location.href=
'https://buy.stripe.com/test_7sY5kF7c7cs98Fv9bV9sk00'"> Pay with UPI / Card
</button><br>
<button id="confirmPaymentBtn">Confirm Payment</button>
<button id="cancelPaymentBtn">Cancel</button> </div> </div>
<script src="script.js">
</script>
</body>
</html>

```

5.2.4 ADMIN PAGE:

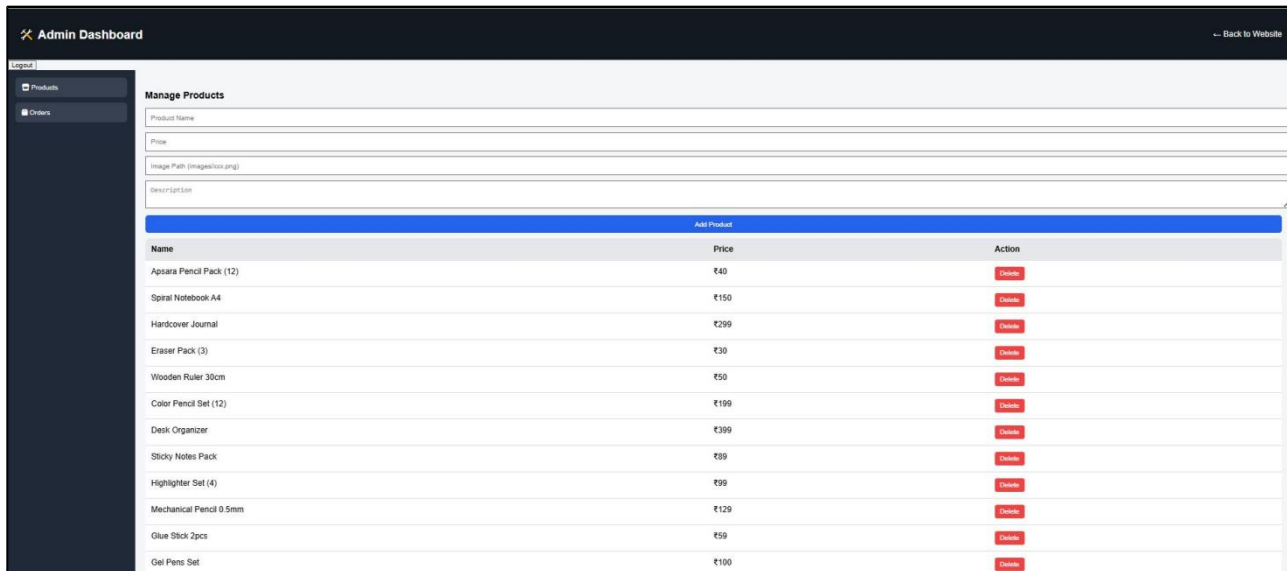


Figure 5.2.4 Admin Page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Admin Panel - Morya Stationery</title>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
<link rel="stylesheet" href="admin.css">
</head>
<body>
<header class="admin-header">
<h2>Admin Dashboard</h2>
<a href="index.html" class="back-site">← Back to Website</a>
</header>
<div class="admin-container">
<aside class="admin-sidebar">
<button onclick="showSection('products')">
<i class="fa fa-store"></i> Products
</button>
<button onclick="showSection('orders')">
<i class="fa fa-box"></i> Orders
</button>
```

```

    </aside>
    <main class="admin-content">

-- PRODUCTS -
    <section id="products" class="admin-section">
    <h3>Manage Products</h3>
    <form id="addProductForm" class="admin-form">
    <input type="text" id="productName" placeholder="Product Name" required>
    <input type="number" id="productPrice" placeholder="Price" required>
    <input type="text" id="productImage" placeholder="Image Path (images/xxx.png)">
    <textarea id="productDesc" placeholder="Description"></textarea>
    <button type="submit">Add Product</button>
    </form>
    <div class="admin-table" id="productTable">
    <div class="table-row header">
    <span>Name</span><span>Price</span><span>Action</span>
    </div>
    </div>
    </section>
    <section id="orders" class="admin-section" style="display:none;">
    <h3>Manage Orders</h3>
    <div class="admin-table" id="orderTable">
    <div class="table-row header">
    <span>Order ID</span>
    <span>Status</span>
    <span>Action</span>
    </div>
    </div>
    </section>
    </main>
    </div>
    <script src="admin.js"></script>
    </body>
    </html>

```

5.2.5 ORDER TRACK:

Morya Stationery Store

[Home](#) | [Products](#) | [My Orders](#) | [Login](#) | [Register](#) |

Track Your Order

Enter Order ID:

Track Order

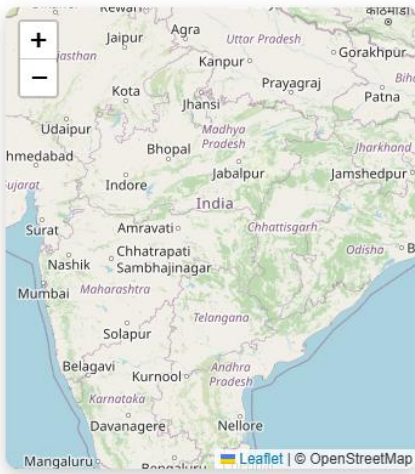


Figure 5.2.5 Order Track Page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Order Tracking - Morya Stationery Store</title>
<link rel="stylesheet" href="styles.css">
<link rel="stylesheet"
href="https://unpkg.com/leaflet/dist/leaflet.css" />
<style> #map {
height: 380px; width: 100%;
```

```

margin-top: 30px;
border-radius: 10px;
box-shadow: 0 6px 15px rgba(0,0,0,0.2); }
#trackingResult {
margin-top: 20px;
padding: 15px;
border-radius: 8px;
background: #f9f9f9;
border-left: 5px solid #2e8b57;
}
</style>
</head>
<body>
<header>
<h1>Morya Stationery Store</h1>
<nav>
<a href="index.html">Home</a>
<a href="products.html">Products</a>
<a href="order.html">My Orders</a>
<a href="ordertrack.html">Track Order</a>
<a href="about.html">About</a>
<a href="contact.html">Contact</a>
<a href="login.html" id="loginLink">Login</a>
</nav>
</header>
<main>
<h2>Track Your Order</h2>
<form id="trackForm">
<label for="orderId">Enter Order ID:</label><br>
<input type="text" id="orderId" name="orderId"
placeholder="e.g. 65af3c2e9d.. required style="width:320px; padding:8px;">

```

```

<br><br>
<button type="submit">Track Order</button>
</form>
<div id="trackingResult"></div>
<div id="map"></div>
</main>
<script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>
<script>
const map = L.map('map').setView([20.5937, 78.9629], 5); // India
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',
  { attribution: '© OpenStreetMap'
}).addTo(map); let marker;
function updateMap(status) { let location;
let message;
if (status === "Order Placed") {
location = [19.0760, 72.8777]; // Store (Mumbai) message = "Order placed at store ";
}
else if (status === "Out for Delivery") {
location = [18.5204, 73.8567]; // Delivery route message = "Out for delivery ";
}
else if (status === "Delivered") {
location = [18.6298, 73.7997]; // Customer message = "Order delivered ✔";
}
if (marker) map.removeLayer(marker); marker = L.marker(location)
.addTo(map)
.bindPopup(message)
.openPopup(); map.setView(location, 12);

```

```

document.getElementById('trackForm').addEventListener('submit', async (e) =>
{ e.preventDefault();
const orderId = document.getElementById('orderId').value.trim();
const resultDiv = document.getElementById('trackingResult');
resultDiv.innerHTML = "🔍 Tracking order...";
try {
const res = await fetch(`/api/order/${orderId}`);
const data = await res.json();
if (!res.ok) {
resultDiv.innerHTML = `<p style="color:red;">${data.message}</p>`;
return; }
resultDiv.innerHTML = `
<p><strong>Order ID:</strong> ${data.orderId}</p>
<p><strong>Status:</strong> ${data.status}</p>
<p><strong>Delivery Address:</strong><br>${data.address}</p>
<p><strong>Order Date:</strong>
${new Date(data.createdAt).toLocaleString()}
</p>.;
updateMap(data.status);
} catch (err)
{ console.error(err);
resultDiv.innerHTML = "❌ Error tracking order.";
} });
</script>
<!-- Keep your common script -->
<script src="script.js">
</script>
</body>
</html>

```


5.3 Testing Approach

Testing is performed using a manual testing approach. Each module is tested individually and then tested again after integration. Different input scenarios such as valid input, invalid input, and empty input are tested to ensure system reliability. Testing is a critical phase in the software development life cycle.

The main objective of testing is to identify errors, defects, and missing requirements in the system and ensure that the software works according to the specified requirements.

In the Stationery E-Commerce System, a manual testing approach is adopted. Each module is tested individually and then tested again after integration to ensure that the complete system functions correctly. Testing is performed at every increment and sprint to maintain quality and reliability.

1. Unit Testing

Unit Testing is performed to test individual modules or components of the system independently.

Each function and API endpoint was tested with valid and invalid inputs to verify correct behavior.

Unit testing was applied to:

- User registration module
- Login module
- Product listing module
- Order placement module

Purpose

- Detect errors at an early stage
- Ensure correctness of individual components

2. Integration Testing

Integration Testing is carried out after unit testing to verify the interaction between different Modules.

Integration testing included :

- Integration of frontend with backend APIs
- Integration of backend with MongoDB database
- Integration of cart module with order module
- This testing ensured that data flows correctly between different modules.

Purpose:

- Identify interface defects
- Ensure smooth data communication between modules

3. System Testing

System Testing is performed on the complete integrated system to verify that it meets the specified requirements. In this project, system testing covered:

- ❖ End-to-end user workflow (Login → Browse Products → Place Order)
- ❖ Order tracking functionality
- ❖ Admin product and order management
- ❖ The system was tested in an environment similar to the real-world scenario.

Purpose:

- Validate the complete system
- Ensure functional correctness

4. Functional Testing

Functional Testing checks whether each function of the application works according to the functional requirements.

Examples from this project:

- Login with valid and invalid credentials
- Display of products from database
- Successful order placement
- Correct display of order status

Purpose:

- Verify functional requirements
- Ensure expected output for given input

5. User Acceptance Testing (UAT)

User Acceptance Testing is performed to verify whether the system is acceptable to end users.

UAT was conducted by testing the application from the user's perspective to ensure:

- Ease of use
- Proper navigation
- Accurate results

Purpose:

- Ensure user satisfaction
- Validate real-world usability

Testing is the process of evaluating software to detect errors and ensure quality. In this project, manual testing was used, including unit testing, integration testing, system testing, functional testing, and user acceptance testing to verify correctness and reliability of the system.

The testing approach adopted for the Stationery E-Commerce System includes Unit Testing, Integration Testing, System Testing, Functional Testing, and User Acceptance Testing.

These testing techniques helped in identifying defects early, improving system reliability, and ensuring that the application meets user requirements.

6. ALPHA BETA Testing

Alpha and Beta testing ensured that the Stationery E-Commerce System was stable, user-friendly, and ready for deployment. Alpha testing helped identify early defects, while beta testing provided valuable user feedback for final improvements

Alpha Testing	Beta Testing
Performed at developer's site	Performed at user's environment
Conducted by developers/testers	Conducted by real users
Before beta testing	After alpha testing

5.3.6 Comparison between Alpha and Beta Testing

5.4 Test Cases

Test Case ID	Module	Test Description	Input Data	Expected Result	Actual Result	Status
TC-01	Registration	Register user with valid details	Valid username, password, address	User registered successfully	User registered	Pass
TC-02	Registration	Register user with existing username	Existing username	Error message displayed	Error displayed	Pass
TC-03	Login	Login with valid credentials	Valid username & password	Login successful	Login successful	Pass
TC-04	Login	Login with invalid credentials	Invalid username/password	Error message displayed	Error displayed	Pass
TC-05	Authentication	Check user session	Logged-in user	User remains logged in	Session maintained	Pass
TC-06	Products	Fetch product list	Request products page	Products displayed from database	Products displayed	Pass
TC-07	Products	Add product to cart	Click “Add to Cart”	Product added to cart	Product added	Pass
TC-08	Orders	Place order	Valid cart items	Order placed successfully	Order placed	Pass
TC-09	Orders	Place order with empty cart	No items in cart	Error message displayed	Error displayed	Pass
TC-10	Orders	View My Orders	Logged-in user	Order history displayed	Orders displayed	Pass
TC-11	Tracking	Track order with valid ID	Valid Order ID	Order status displayed	Status displayed	Pass
TC-12	Tracking	Track order with invalid ID	Invalid Order ID	Error message displayed	Error displayed	Pass
TC-13	Admin – Products	Add new product	Product details	Product added to database	Product added	Pass
TC-14	Admin – Products	Delete product	Product ID	Product removed	Product removed	Pass
TC-15	Admin – Orders	Update order status	Order ID, new status	Status updated	Status updated	Pass
TC-16	Logout	Logout user	Logout Request	User Logout	Logged Out	Pass

5.4 Test Cases Table

CHAPTER 6 : RESULTS AND DISCUSSION

6.1 Test Reports

This test report describes the testing activities carried out for the Stationery E-Commerce System. The purpose of this report is to summarize the testing process, test results, and overall system quality after executing various test cases. Testing was conducted to ensure that the system meets the specified functional and non-functional requirements and works correctly under different scenarios.

“Both positive and negative test cases were executed to verify system behavior.”

Parameter	Description
Operating System	Windows 10
Frontend	HTML, CSS, JavaScript
Backend	Node.js, Express.js
Database	MongoDB
Browser	Google Chrome

6.1.1 Test Reports Table

Test Execution Summary

All planned test cases were executed manually. The test cases included functional testing, integration testing, system testing, and user acceptance testing.

Description	Count
Total Test Cases Designed	16
Test Cases Executed	16
Test Cases Passed	16
Test Cases Failed	0

6.1.2 Test Execution Table

The results of the testing process are as below :

- All major functionalities of the system worked as expected.
- User registration and login modules functioned correctly.
- Products were successfully fetched from the database and displayed on the user interface.
- Orders were placed and stored correctly in the database.
- Order tracking displayed accurate order status.
- Admin functionalities such as adding products and updating order status worked properly.
- No critical defects were found during testing.

Defects Identified:

During the testing phase, minor issues related to UI alignment and input validation were identified and resolved. No major functional defects were observed in the final system.

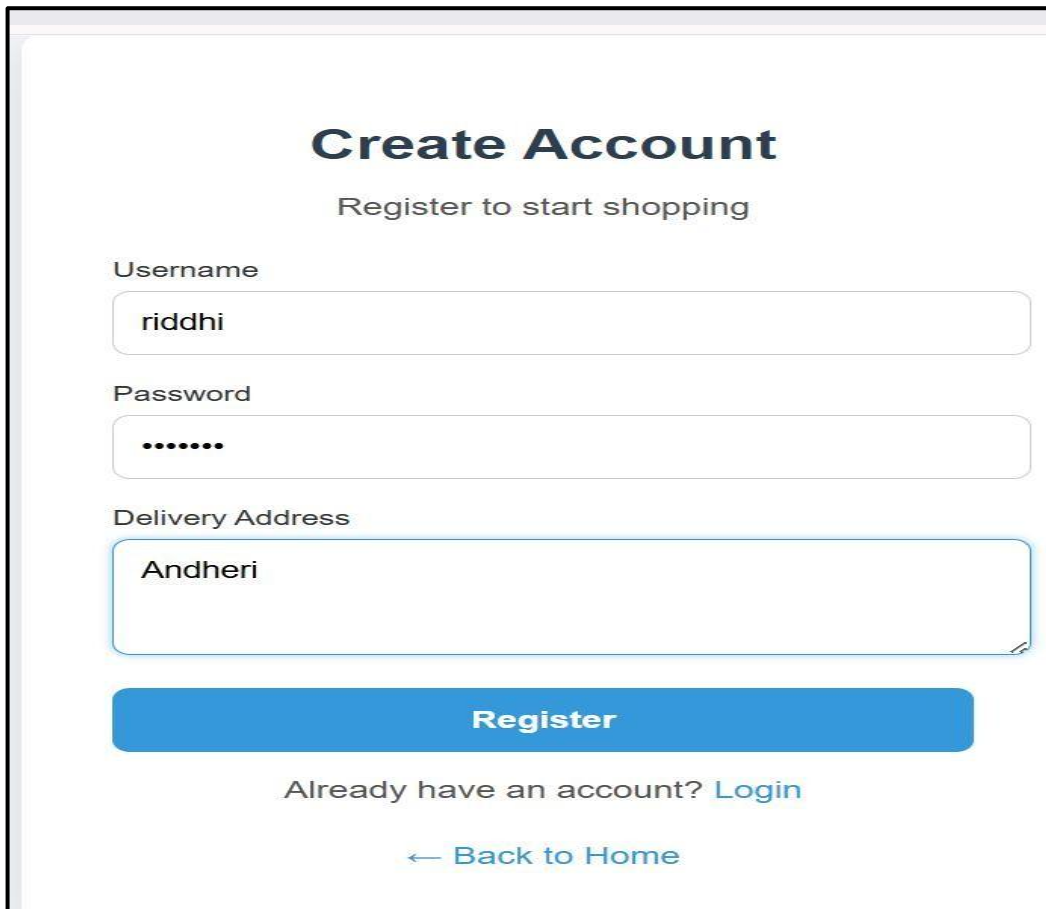
User Acceptance Testing Result

- The system was tested from the end-user perspective to ensure usability and ease of navigation. Users were able to:
- Register and login without difficulty
- Browse products easily
- Place orders successfully
- Track orders accurately

The system was found to be user-friendly and acceptable for deployment.

6.2 User Manual

1.Registration Page



Create Account

Register to start shopping

Username

riddhi

Password

.....

Delivery Address

Andheri

Register

Already have an account? [Login](#)

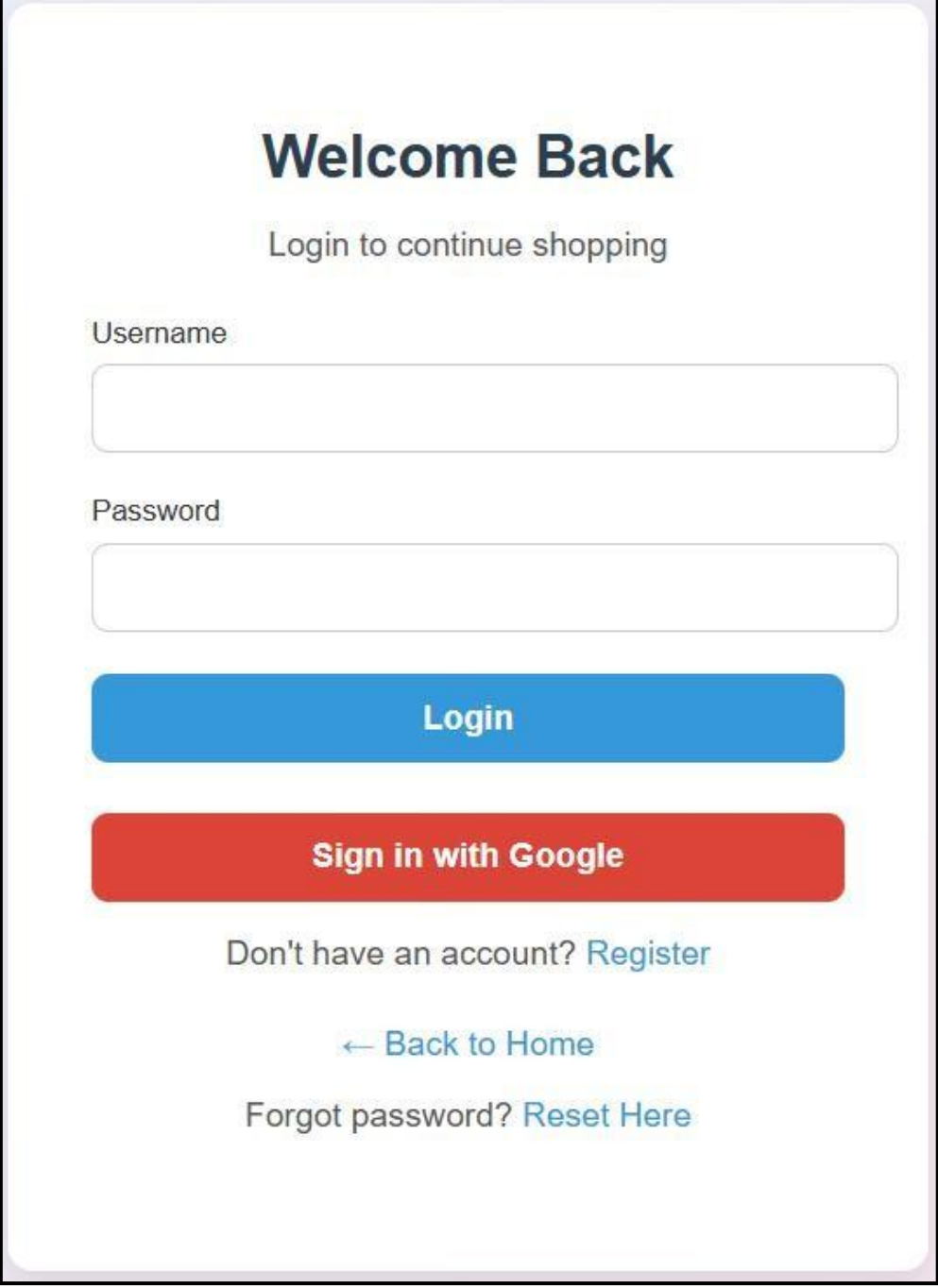
[← Back to Home](#)

Figure 6.2.1 Register Page

Steps to Register:

1. Open the home page of the Stationery E-Commerce website.
2. Click on the Register button.
3. Enter required details such as username, password, and address.
4. Click on the Register button.
5. Redirects to Login Page .

6.2.2 Login Page



The image shows a login page with a light gray background. At the top, the text "Welcome Back" is displayed in a large, bold, dark blue font. Below it, the text "Login to continue shopping" is in a smaller, regular dark blue font. There are two input fields: "Username" and "Password", both with light gray borders and rounded corners. Below the "Password" field is a blue "Login" button with white text. Underneath the button is a red "Sign in with Google" button with white text. At the bottom, there are three links: "Don't have an account? Register" (with "Register" in blue), "← Back to Home" (with "Back to Home" in blue), and "Forgot password? Reset Here" (with "Reset Here" in blue).

Welcome Back

Login to continue shopping

Username

Password

Login

Sign in with Google

Don't have an account? [Register](#)

[← Back to Home](#)

Forgot password? [Reset Here](#)

Figure 6.2.2 Login Page

Steps for Login :

There are multiple Options for Users to Authenticate through Registration and Email Sign up Verification with directing it to Login Page as Login is mandatory to validate users and store information and orders into Database .

Steps to Login:

1. Click on the Login option.
2. Enter username and password / sign up with google option.
3. Click on the Login button.
4. On successful login, the user is redirected to the home page.
5. Now Users can buy products and check Orders history.

6.2.3 Products Page

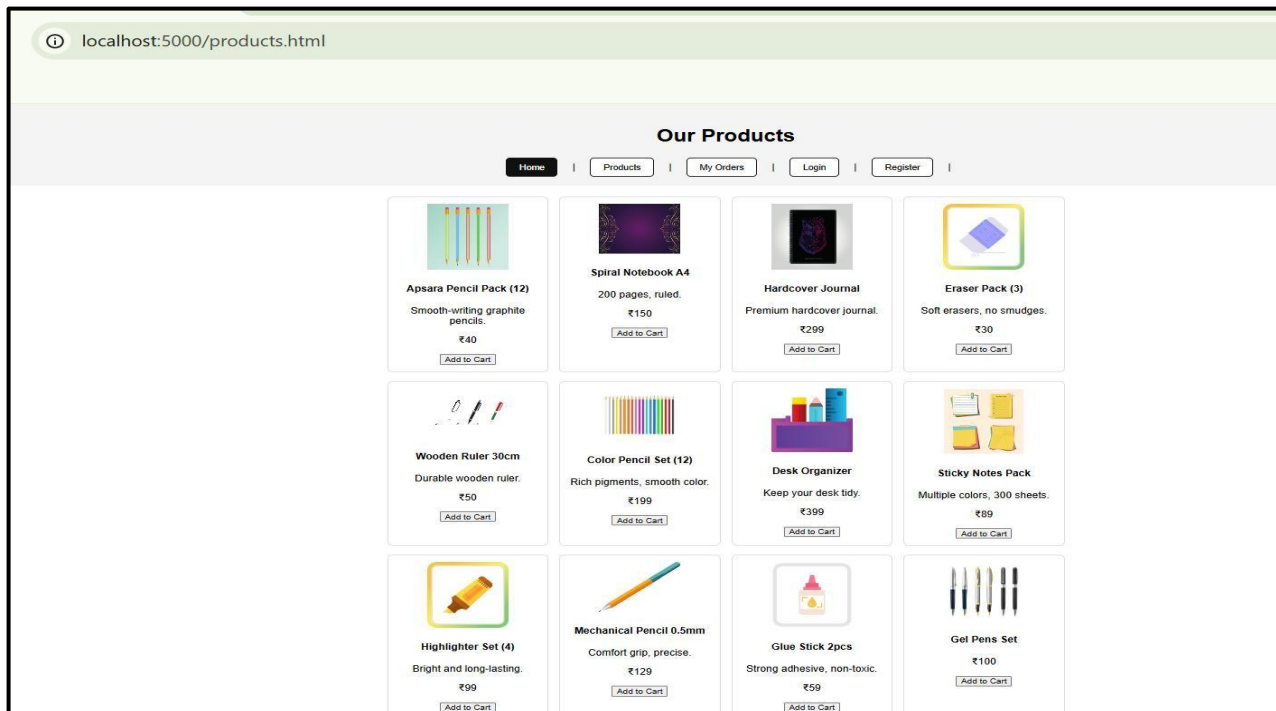


Figure 6.2.3 Product Page

Steps to Browse Products:

1. Click on the Products tab from the navigation menu.
2. The list of available stationery products and Print upload is displayed.
3. Users can view product details such as name, price, and image

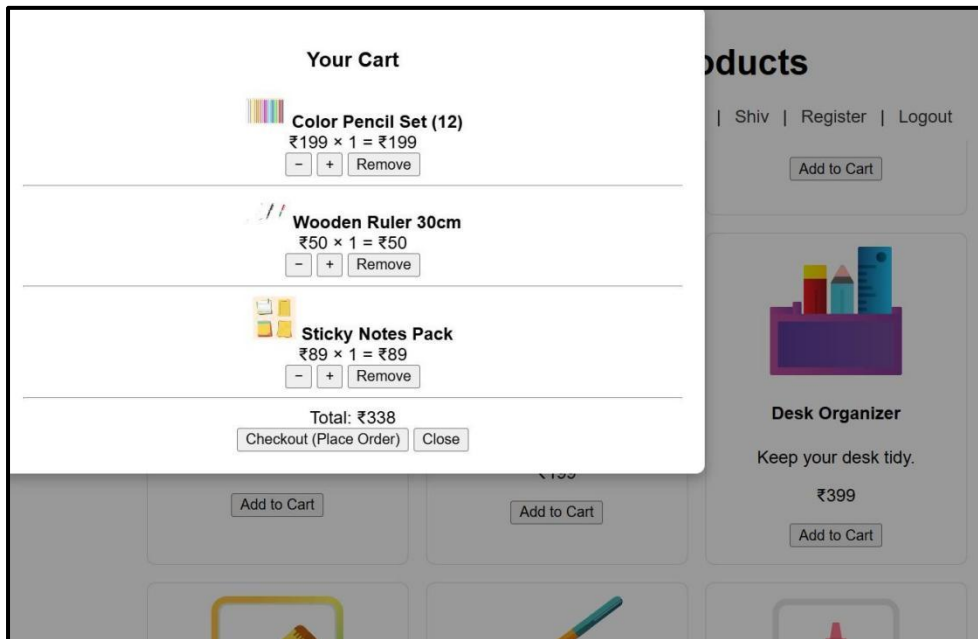


Figure 6.2.3 Cart Page

Steps to Add Products to Cart:

1. Select a product from the product list.
2. Click on the Add to Cart buytton.
3. The product is added to the cart.
4. Cart count is updated Automatically.

6.2.4 CheckOut Page

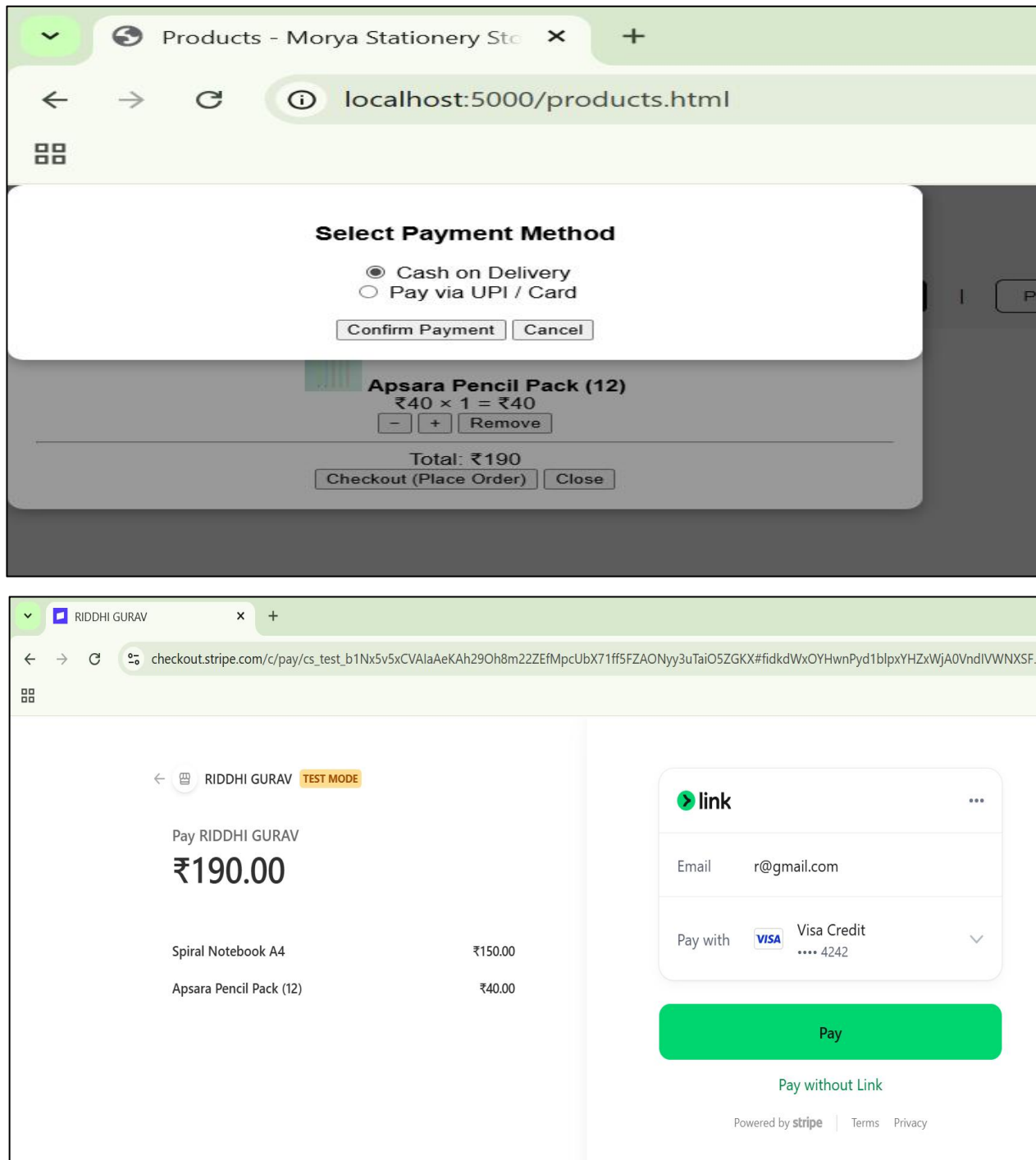


Figure 6.2.4 Checkout Page

Steps to Place an Order

1. Click on the View Cart button.
2. Review selected products.
3. Click on Checkout.
4. Select payment method COD / UPI.
5. Confirm the order.
6. Order is placed successfully.

6.2.5 Orders Page

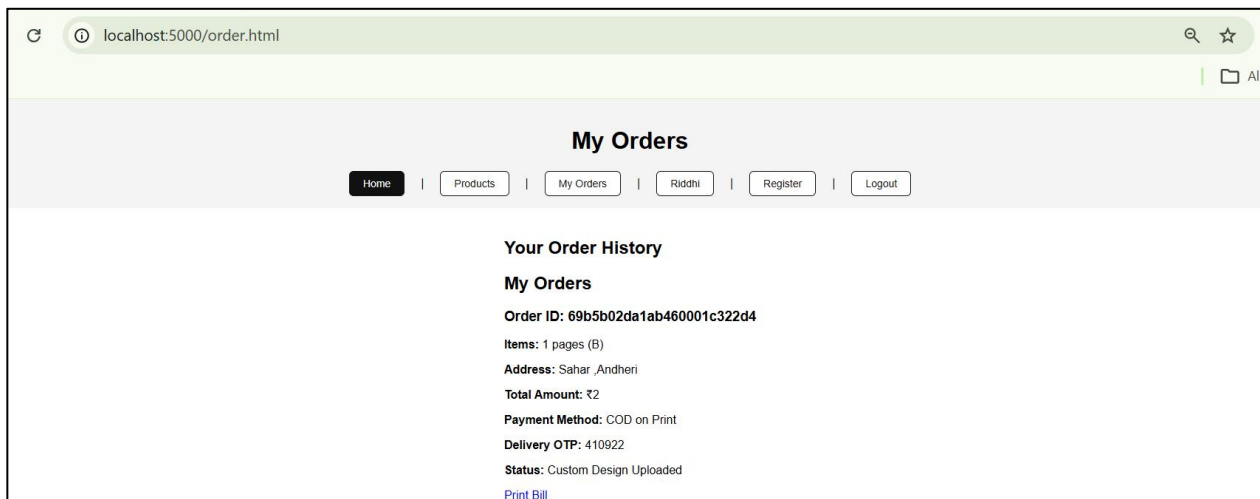


Figure 6.2.5 Orders Page

Steps to View Orders:

1. Click on Orders → My Orders.
2. The list of previously placed orders is displayed.
3. Users can view order details and order status.

6.2.6 Tracking Page

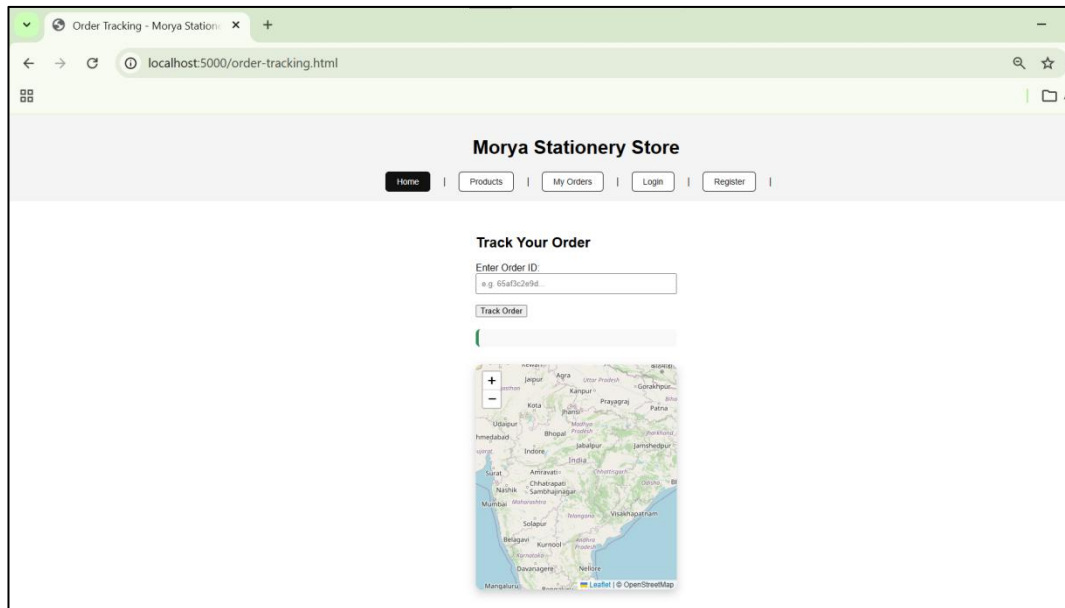


Figure 6.2.6 Tracking Page

Steps to Track an Order :

1. Click on Orders → Track Order.
2. Enter the Order ID.
3. Click on Track.
4. Current order status is displayed

6.2.7 Admin Panel Page

Admin Dashboard [← Back to Website](#)

Manage Products

Product Name

Price

Image Path (images/xxx.png)

Description

[Add Product](#)

Name	Price	Action
Apsara Pencil Pack (12)	₹40	Delete
Spiral Notebook A4	₹150	Delete
Hardcover Journal	₹299	Delete
Eraser Pack (3)	₹30	Delete
Wooden Ruler 30cm	₹50	Delete
Color Pencil Set (12)	₹199	Delete

Figure 6.2.7 Admin Panel

Steps to Manage Products:

1. Login to Admin Panel.
2. Click on Products.
3. Add new product by entering details.
4. Edit or delete existing products.
5. Click on Orders to update Order status
6. Check Order information

6.2.8 Log Out

1. Click on the Logout option.
2. User session is terminated.
3. User is redirected to the login page.

Chapter 7 : CONCLUSION

7.1 Conclusion

The Stationery E-Commerce System developed in this project provides a complete web-based solution for buying stationery products online. The system successfully integrates frontend and backend technologies to deliver a smooth and user-friendly experience. Users can register, login, browse products, add items to the cart, place orders, and track their orders efficiently.

The administrator can manage products and orders through a dedicated admin panel, ensuring effective control over inventory and order processing. The project fulfills all the objectives defined during the planning phase and demonstrates practical implementation of modern web technologies such as Node.js, Express.js, MongoDB, HTML, CSS, and JavaScript. The use of Agile methodology enabled incremental development and continuous improvement of the system.

7.2 Limitations of the System

Despite successful implementation, the system has certain limitations:

1. Internet Dependency

The system requires a stable internet connection to function, limiting access in areas with poor connectivity.

2. Limited Payment Options

Currently, the system supports basic payment options like Cash on Delivery (COD) and a dummy card input format. Real-time payment gateway integration such as UPI, Stripe, Razorpay, PayPal is not fully implemented. No refund policy included.

3. Limited Analytics

The product management system is simple and can be enhanced with advanced inventory and analytics features.

4. No Mobile Application

The project is only a web-based system. There is no dedicated mobile application for Android or iOS users.

7.3 Future Scope

The system can be enhanced in the future by implementing the following features :

- **Online Payment Gateway Integration**

Integration with UPI, credit/debit cards, and digital wallets.

- **Mobile Application Development**

Development of Android and iOS applications for better accessibility.

- **Admin Analytics Dashboard**

Graphical representation of sales, orders, and user activity.

REFERENCES

Sr. No.	Tool / Website / Application	Reference Link
1	Visual Studio Code (VS Code)	https://code.visualstudio.com/
2	HTML5 Standard	https://html.spec.whatwg.org/
3	CSS Reference (MDN)	https://developer.mozilla.org/en-US/docs/Web/CSS
4	JavaScript Reference (MDN)	https://developer.mozilla.org/en-US/docs/Web/JavaScript
5	Font Awesome	https://fontawesome.com/
6	Node.js	https://nodejs.org/
7	Express.js	https://expressjs.com/
8	MongoDB	https://www.mongodb.com/
9	MongoDB Compass	https://www.mongodb.com/products/compass
10	Mongoose ODM	https://mongoosejs.com/
11	bcrypt	https://www.npmjs.com/package/bcrypt
12	express-session	https://www.npmjs.com/package/express-session
13	CORS Package	https://www.npmjs.com/package/cors

14	Axios	https://www.npmjs.com/package/axios
15	Render	https://render.com/
16	Git	https://git-scm.com/
17	GitHub	https://github.com/
18	Postman	https://www.postman.com/
19	Firebase	https://firebase.google.com/
20	Stripe Payment Gateway	https://stripe.com/
21	UPI Payment Reference (NPCI)	https://www.npci.org.in/what-we-do/upi/product-overview
22	Agile Model Reference	https://www.atlassian.com/agile
23	Scrum Model Reference	https://www.scrum.org/resources/what-is-scrum
24	Incremental Model Reference	https://www.geeksforgeeks.org/software-engineering-incremental-process-model/
25	MDN Web Docs	https://developer.mozilla.org/
26	W3Schools	https://www.w3schools.com/